LuoSA 用户手册

企业级 RAG + Agent 平台

LuoSA 团队

2026-09-30

- LuoSA 用户手册
  - 手册目录
    - 通用（所有角色必读）
    - 按角色分章（核心交付 —— 按您的角色阅读）
    - 功能模块参考（按需查阅）
  - 角色矩阵（速查）
  - 角色权限详解
    - Viewer（访客）—— 阅读者
    - Contributor（编辑）—— 创作者
    - Admin（租户管理员）—— 基础设施运维
    - Owner（所有者）—— 空间负责人
    - System Admin（平台系统管理员）—— 跨租户超管
  - 跨租户共享与 API Key
  - 关键不变式（手册通则）
  - 阅读路径建议
  - 反馈与修订
- 01 · 平台概览
  - 1. LuoSA 是什么

2. 核心能力地图
2.1 三类核心使用方式
2.2 七大功能模块
2.3 三种知识库类型
3. 典型场景
4. 部署形态
5. 登录入口与首屏
6. 关键概念（先认识这些）
7. 与角色相关的能力边界
8. 阅读建议
02 • 快速上手
1. 登录
1.1 普通登录
1.2 通过邀请链接登录
1.3 OIDC / SSO 登录
2. 主界面速览
3. 与默认智能体对话（3 步）
第 1 步：进入新建对话首页
第 2 步：选择 Agent（可选）
第 3 步：提问
4. 浏览一个知识库
第 1 步：进入知识库列表
第 2 步：选择一个 KB
第 3 步：检索测试
5. 看一个智能体
第 1 步：进入智能体列表
第 2 步：选择一个 Agent
第 3 步：点击 Agent 卡片 → 与它对话
6. 邀请收件箱
7. 个人资料
8. 5 分钟清单（跟完即上手）
9. 下一步

角色手册 • Viewer（访客 / 业务人员）
1. 您的能力地图
✅ 您能做的事
❌ 您不能做的事
2. 典型工作流
场景 A：检索一个已有 KB
场景 B：与 Agent 对话
场景 C：接受一个邀请
场景 D：查看工作区成员
3. 关键页面详解
3.1 对话主界面（`/platform/chat/:chatid`）
3.2 知识库详情（`/platform/knowledge-bases/:kbId`）
3.3 Agent 列表（`/platform/agents`）
4. 常见问题（FAQ）
Q1：为什么我看不到「新建对话」按钮?
Q2：为什么我能跑某个 Agent 但跑不了另一个?
Q3：我能下载 Agent 引用的原文吗?
Q4：为什么我看到的是「待审批」状态?
Q5：能在移动端用吗?
5. 何时联系 Owner
6. 进一步阅读
角色手册 • Contributor（编辑）
1. 您的能力地图
✅ 您能做的事
⚠️ 部分能做（条件性）
❌ 您不能做的事
2. 典型工作流
场景 A：建立您的第一个知识库
场景 B：上传文档到 KB
场景 C：调 KB 解析参数
场景 D：创建您的第一个 Agent
场景 E：把 Agent 标记为 Viewer 可跑

场景 F：复制别人的 KB / Agent

3. 关键页面详解

3.1 KB 列表（`/platform/knowledge-bases`）

3.2 KB 详情（`/platform/knowledge-base/:kbId`）— 您作为创建者

3.3 Agent 编辑器（`/platform/agents` 详情）

4. 创建资源的「标准流程」清单

5. 常见问题（FAQ）

Q1：为什么我编辑不了别人的 KB?

Q2：文档上传后一直「解析中」?

Q3：我能修改工作区的全局模型配置吗?

Q4：为什么我新建 Agent 时某些工具看不到?

Q5：怎么把 Agent / KB 共享给其他工作区?

6. 何时升级到 Admin

7. 进一步阅读

角色手册 • Admin（租户管理员）

1. 您的能力地图

✅ 您能做的事

⚠️ 部分能做（条件性）

❌ 您不能做的事

2. 待审批用户流程（高频操作）

第 1 步：进入待审批队列

第 2 步：审查申请

第 3 步：批准 / 拒绝

3. 配置基础设施的标准流程

场景 A：接入新的 LLM（OpenAI / DeepSeek / Claude / 国产模型…）

场景 B：配置向量库

场景 C：添加 MCP 服务

场景 D：安装工作区 Skill

场景 E：配置沙箱

场景 F：配置 IM 通道

场景 G：配置数据源

4. 审计日志

5. 评估（Evaluation）
6. 任务队列与 Worker Pool 治理
7. 关键页面详解
7.1 设置模态（统一入口）
7.2 System Admin 专属 section（**Admin 看不到**）
8. 常见问题（FAQ）
Q1：为什么我点不动「成员」管理?
Q2：为什么我看不到「平台 API Key」入口?
Q3：我能调整 KB 的解析参数吗?
Q4：审批完用户后，他还需要做什么?
Q5：MCP 服务的 OAuth2 中途授权怎么处理?
9. 何时升级到 Owner
10. 进一步阅读
角色手册 • Owner（所有者）
1. 您的能力地图
✅ 您能做的事
❌ 您不能做的事
2. 邀请新成员
场景 A：邀请一个已注册用户
场景 B：创建 share-link（让对方自服务注册）
场景 C：撤销未接受的邀请
3. 修改成员角色
4. 移除成员
5. 修改空间配置
6. 租户 API Key 管理
7. 跨工作区共享（Organization）
8. 转让所有权
9. 删除空间
10. 关键页面详解
10.1 成员管理（`/platform/settings?section=members`）
10.2 空间信息（`/platform/settings?section=tenant`）

11. 常见问题（FAQ）
Q1：为什么我点不动「删除空间」按钮?
Q2：share-link 有有效期吗?
Q3：通过 share-link 加入的用户需要审批吗?
Q4：API Key 泄露了怎么办?
Q5：我能把 Owner 转给不在本工作区的用户吗?
Q6：Owner 能退出自己的空间吗?
12. 工作流检查清单（季度）
13. 进一步阅读
角色手册 • System Admin（平台系统管理员）
1. 您的能力地图
✅ 您能做的事
⚠️ 与 Owner 的边界
2. 平台管理员入口
3. 提升 / 撤销其他 System Admin
提升
撤销
列出所有
4. 平台 API Key
5. 运行时队列与 Worker Pool 治理
第 1 步：进入运行时队列
第 2 步：查看池状态
第 3 步：per-model 并发治理
第 4 步：查看任务
6. 平台审计日志
7. 用户管理与审批
创建用户
重置密码
跨租户审批
8. 平台系统设置
9. 关键不变式（System Admin 特别版）

10. 关键页面详解
10.1 平台设置（`/platform/system/system-global`）
10.2 运行时队列（`/platform/system/runtime-queues`）
10.3 待审批用户（`/platform/system/pending-users` 或工作区级）
11. 常见问题（FAQ）
Q1：为什么我切到工作区后看不到「成员管理」按钮?
Q2：我能把其他租户的 Owner 降级吗?
Q3：平台 API Key 和租户 API Key 能混用吗?
Q4：怎么把 System Admin 标志位从自己账号上移除?
Q5：运行时队列里任务一直 pending 怎么办?
12. 应急操作清单（事故响应）
13. 进一步阅读
模块参考 • 知识库（Knowledge Base）
1. 知识库的三种类型
2. 创建知识库
路径
表单字段
3. KB 详情页布局
4. 上传与解析
4.1 上传源
4.2 解析流程
4.3 支持的文档格式（DocReader）
4.4 特殊能力
4.5 排错
5. KB 设置的 13 个子面板
6. 检索原理（`HybridSearch`）
7. 数据源
支持的数据源
配置步骤
8. FAQ 维护
9. 知识图谱（Graph）

10. 标签与文件夹
标签
文件夹
11. 共享
工作区内共享
跨工作区共享
12. 复制 / 收藏 / 删除
13. 关键不变式
14. 进一步阅读
模块参考 · 智能体（Agent）
1. 智能体的概念
2. 内置智能体（`config/builtin_agents.yaml`）
3. Agent 类型与预设（`config/agent_type_presets.yaml`）
4. Agent 编辑器（左导航分组）
5. Agent 能调用的工具（`internal/agent/tools/`）
5.1 知识检索
5.2 文件操作
5.3 Wiki
5.4 数据
5.5 多模态
5.6 其他
6. Agent 类型与 Skill / Sandbox 集成
SkillsEnabled
SkillsSelectionMode
Sandbox
7. Agent 服务核心流程（`CreateAgentEngine`）
8. 共享 Agent
9. Agent 的运行生命周期
10. 调试与测试
10.1 Agent 编辑器内测试
10.2 单步调试
10.3 Langfuse 集成（观测性）

11. 自定义 Agent 的最佳实践
12. 关键不变式
13. 进一步阅读

模块参考 · 工作流（Workflow）
1. 工作流是什么
2. 编辑器（`/platform/workflow/:id/edit`）
2.1 工具栏
2.2 节点组件（`workflow/components/`）
3. DSL（领域特定语言）
3.1 结构（v1）
3.2 校验规则
3.3 导入 / 导出
4. 发布模型
4.1 发布流程
4.2 状态切换
5. 运行与调度
5.1 运行
5.2 单节点调试
5.3 调度（Cron）
5.4 Checkpoint（断点）
5.5 事件流（Redis）
6. 节点适配器
7. 节点配置（典型）
7.1 LLM Node
7.2 Retrieval Node
7.3 Switch Node
7.4 Code Node
7.5 HTTP Node
8. 关键 API
9. 最佳实践
10. 关键不变式
11. 进一步阅读

模块参考 · 会话与聊天（Session / Chat）
1. 概念
2. 三类入口
3. 新建对话首页（`/platform/creatChat`）
4. 对话主界面（`/platform/chat/:chatid`）
4.1 区域
4.2 关键元素
5. 会话后台管线
5.1 知识问答管线（`session_knowledge_qa.go`）
5.2 智能体问答管线（`session_agent_qa.go`）
5.3 临时附件（`session_attachment_staging.go`）
5.4 沙箱绑定（`session_sandbox_pin.go`）
6. 会话主体治理（关键不变式）
7. 会话操作
8. 跨主体通道
8.1 Embed 网站 widget（`/embed.html`）
8.2 IM 集成（10 个平台）
8.3 REST API
9. 推荐问题（来源）
10. 长期记忆（Memory）
11. 聊天历史设置
12. 关键不变式
13. 进一步阅读
模块参考 · 技能与 MCP（Skills / MCP）
1. Skills（技能）
1.1 概念
1.2 SKILL.md 格式

LuoSA 用户手册

欢迎使用 **LuoSA** —— 企业级 RAG + Agent 平台。本手册面向不同角色的用户，按**权限分层**组织内容：先看总览明确能力边界，再按您所属角色阅读对应章节。

> LuoSA 把「文档解析 → 知识库 → Agent 推理 → 工作流编排」做成可自托管的开源平台。后端 Go，前端 Vue3，文档解析 Python gRPC（DocReader），MCP 服务器 Node。

手册目录

通用（所有角色必读）

- 01 平台概览 — LuoSA 是什么、能做什么、典型使用场景
- 02 快速上手 — 5 分钟从登录到第一次对话

按角色分章（核心交付 —— 按您的角色阅读）

角色	文档	一句话定位
Viewer（访客 / 业务人员）	roles/01-viewer.md	只读、对话、跑公开 Agent
Contributor（编辑）	roles/02-contributor.md	创建知识库与智能体
Admin（租户管理员）	roles/03-admin.md	配置租户基础设施
Owner（所有者）	roles/04-owner.md	管理成员与空间
System Admin（平台系统管理员）	roles/05-system-admin.md	平台级运维

功能模块参考（按需查阅）

模块	文档	适用范围
知识库（KB）	modules/01-knowledge-base.md	Contributor+
智能体（Agent）	modules/02-agent.md	Contributor+
工作流（Workflow）	modules/03-workflow.md	Contributor+
会话与聊天（Session / Chat）	modules/04-session-chat.md	所有角色
技能与 MCP（Skills / MCP）	modules/05-skills-mcp.md	Admin+
系统设置（Settings）	modules/06-system-settings.md	视 section 而定
Wiki 模式	modules/07-wiki.md	Contributor+

角色矩阵（速查）

LuoSA 使用「**角色 → 路由门**」强矩阵模式（`internal/router/rbac.go`）。租户内 4 级角色，平台级 2 个超管标志位。下表每一行都对应一个后端中间件强制门。

能力	Viewer	Contributor	Admin	Owner	System Admin[1]
浏览 / 检索 KB 与 Agent	✅	✅	✅	✅	✅
聊天检索（RAG / Agent）	✅	✅	✅	✅	✅
运行 `runnable_by_viewer=true` 的 Agent	✅	✅	✅	✅	✅
接收邀请 / 主动加入空间	✅	✅	✅	✅	—
创建 KB / Agent（自己名下）	❌	✅	✅	✅	✅
编辑他人 KB / Agent	❌	❌	✅	✅	✅
配置基础设施[2]	❌	❌	✅	✅	✅
批准待审批用户	❌	❌	✅	✅	✅
查看审计日志	❌	❌	✅	✅	✅
邀请 / 移除 / 修改成员	❌	❌	❌	✅	✅
修改空间配置 / 租户 API Key	❌	❌	❌	✅	✅
转让所有权 / 删除空间	❌	❌	❌	✅	✅[3]
提升 / 撤销系统管理员	❌	❌	❌	❌	✅
平台 API Key / 运行时队列 / 平台系统设置	❌	❌	❌	❌	✅

[1] System Admin 是与租户角色**正交**的标志位（`users.is_system_admin`）。切换到任意租户时被授予**临时 Admin**（不写入 `tenant_members`），不受租户 4 级层级限制。 [2] 基础设施 = 模型 / 向量库 / MCP / 沙箱 / Skill / 解析器 / 存储 / IM 通道 / 长期记忆 / Web 搜索。 [3] System Admin 跨租户访问需要同时打开用户级 `can_access_all_tenants` 与集群级 `enable_cross_tenant_access`。

> ⚠️ **重要**：本手册中所有「角色能做什么」均与后端中间件 `RequireRole / RequireOwnershipOrRole / RequireSystemAdmin` 严格对齐。**前端 UI 仅作展示，最终授权以后端为准**——前端按钮可见并不等于一定能成功调用接口。

角色权限详解

Viewer（访客）—— 阅读者

最低权限角色。只能读，不能创建。

- **典型用户**：业务分析师、客服、市场调研、销售支持、新员工（只读访问部门资料）。
- **典型场景**：检索已有知识库、与 Bot 对话、跑已经发布的 Agent、查看成员名单与审计日志。
- **不能做的事**：创建 KB / Agent、修改任何配置、上传文档、邀请成员。
- 详见 roles/01-viewer.md。

Contributor（编辑）—— 创作者

可创建资源并编辑自己创建的资源。

- **典型用户**：领域专家、产品经理、运营、HR（写自己负责的 KB）。
- **典型场景**：建立某个产品线的 KB、写一个客服 FAQ Agent、上传并解析文档、配置 KB 级别的解析/切片/向量库。
- **不能做的事**：编辑他人资源、修改基础设施、邀请成员、配置模型/MCP/IM。
- 详见 roles/02-contributor.md。

Admin（租户管理员）—— 基础设施运维

管理租户的所有基础设施配置。

- **典型用户**：IT 运维、平台管理员（公司内部）、算法工程师。
- **典型场景**：接入 LLM API、配置向量库、配置 IM 集成、管理 MCP 服务、批准待审批用户、查看审计日志。
- **不能做的事**：邀请成员、修改空间配置、管理租户 API Key、删除空间。
- 详见 roles/03-admin.md。

Owner（所有者）—— 空间负责人

完全控制租户，包括成员与空间配置。

- **典型用户**：团队负责人、部门主管、空间创建者。
- **典型场景**：邀请新成员、修改成员角色、创建 share-link、管理租户 API Key、修改空间名与配置、删除空间。
- 详见 roles/04-owner.md。

System Admin（平台系统管理员）—— 跨租户超管

与租户角色正交，平台级运维。

- **典型用户**：平台运营、SRE、安全审计。
- **典型场景**：审批系统级设置、管理平台 API Key、监控系统队列、平台级审计、跨租户切换。
- 详见 roles/05-system-admin.md。

跨租户共享与 API Key

LuoSA 除了人类用户，还有两个独立的鉴权维度：

- **跨工作区（Organization）**：把 KB 共享给组织时按 `OrgMemberRole`（admin / editor / viewer）裁剪权限。这是**与租户角色独立**的第二层 RBAC，由 `internal/types/organization.go` 定义。
- **API Key 主体**：不属于人类用户，携带 `APIKeyCapability` 列表与 KB allow-list。默认拒绝未在 `apiKeyRouteAuthorizer` 注册的路由（声明式 + 默认拒绝）。详见 modules/06-system-settings.md。

关键不变式（手册通则）

1. **最后一位 Owner 不可降级**：尝试把最后一位 active Owner 改成非 Owner 会返回 `409 + ErrLastOwner`。
2. **跨租户访问需要双向开启**：用户级 `can_access_all_tenants` 与 集群级 `enable_cross_tenant_access` 都为 true 才生效。
3. **`enable_rbac=false` 只对租户角色门生效**：`RequireSystemAdmin` 与 API Key 门不受影响（rollout 安全窗口）。
4. **孤儿租户自愈**：当租户无任何 active 成员、用户登录的是自己的 home tenant 时，自动晋升为 Owner（迁移 000043）。
5. **公开注册默认未审批**：自注册用户默认 `is_approved=false`，必须由 Admin+ 在「待审批」队列中批准后才能登录；OIDC 用户免审批。
6. **Owner 不能退出自己创建的空间**（如果自己是最后一位 Owner）。
7. **共享 Agent 的写入工具被过滤**：跨租户共享的 Agent 会自动过滤写操作工具，确保只读语义。

阅读路径建议

您是谁	推荐阅读顺序
第一次使用 / 业务人员	01 → 02 → roles/01-viewer
团队内创建者	01 → 02 → roles/02-contributor → modules/01 + 02
租户管理员 / IT	01 → 02 → roles/03-admin → modules/06
空间 Owner	01 → roles/04-owner
平台系统管理员	01 → roles/05-system-admin → modules/06

反馈与修订

- 仓库 `docs/user-manual/` 目录下编辑对应 Markdown 文件。
- 角色权限矩阵与代码强对齐，**修改前请先确认后端中间件（`internal/router/rbac.go` 与 `internal/middleware/rbac.go`）已同步**。
- 提交前跑一遍 `make lint` 与 `make test`。

01 • 平台概览

> 本文是**所有角色必读**。先理解 LuoSA 能做什么，再去读对应角色的章节。

1. LuoSA 是什么

LuoSA 是一个**企业级 RAG + Agent 框架**。它的核心是把以下四件事串成一个可自托管的开源平台：

```
文档解析 → 知识库 → Agent 推理 → 工作流编排
```

能力	通俗解释
RAG 检索	把您公司的文档丢进去，用户提问时从文档里找最相关的段落给 LLM 看
ReAct Agent	LLM 不光答，还能看 LLM 自己调用工具（查 KB / 跑脚本 / 调 MCP / 检索数据库）
工作流编排	把多个 LLM 调用、检索、代码、Agent 节点拼成可视化 DAG，按发布版本运行
Skills / MCP	给 Agent 装上预置技能和外部 MCP 服务，让它能写脚本、操作第三方系统

LuoSA 不是一个「聊天机器人」。它是一套让企业把**私有知识 + 业务流程**接进 LLM 的完整方案。

2. 核心能力地图

2.1 三类核心使用方式

入口	说明	适合谁
Web 控制台	浏览器访问主界面（Vue3 SPA），所有功能都有 UI	所有角色
REST / gRPC API	程序化访问，LuoSA 提供 API Key 主体鉴权	集成开发者
IM 渠道	飞书 / 企微 / Slack / Telegram 等 10 个平台	业务人员对话

2.2 七大功能模块

模块	一句话	主要路径
知识库（KB）	上传文档、解析、切片、检索	`/platform/knowledge-bases`
智能体（Agent）	单轮 ReAct 推理 + 工具调用	`/platform/agents`
工作流（Workflow）	多节点编排（Dify-parity）	`/platform/workflow`
会话（Session）	聊天主界面 + 历史 + IM 接入	`/platform/chat/:chatid`
技能（Skills）	预置脚本 + 沙箱执行	设置 → Skills
MCP 服务器	接入外部 MCP 协议服务	设置 → MCP
Wiki 模式	把 KB 蒸馏成结构化、可互链的 Markdown 知识库	KB 详情 → Wiki 标签
长期记忆（Memory）	跨会话记忆用户偏好 / 事实	设置 → 记忆

2.3 三种知识库类型

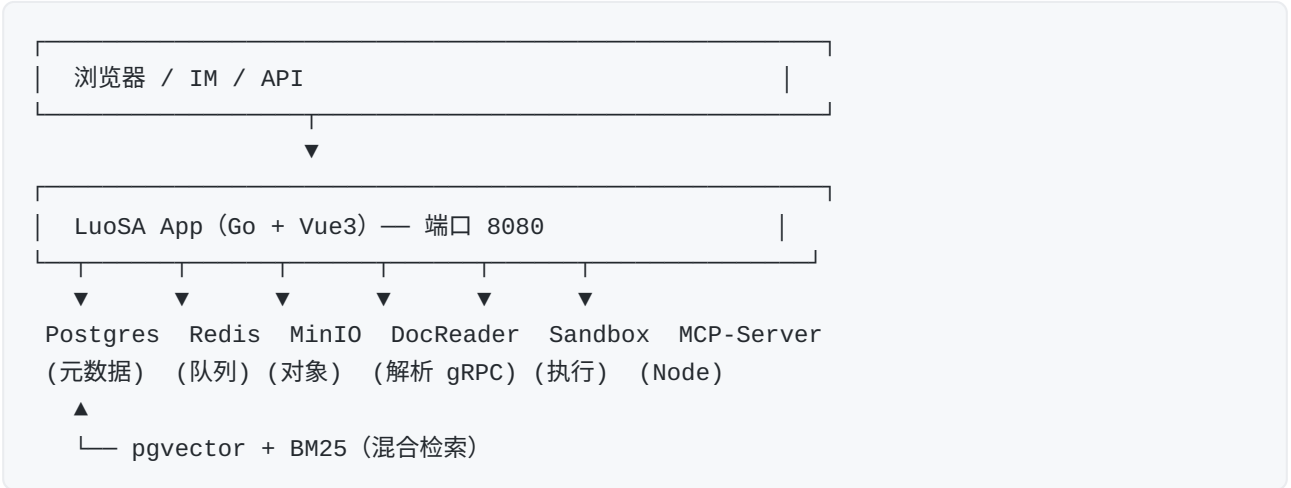
类型	用途	特点
Document（文档库）	上传 PDF / Word / Excel / 图片 / HTML / EPUB / XMind 等	自动解析 + 切片 + 检索
FAQ（问答库）	一问一答的标准问答对	精确匹配优先
Wiki（Wiki 库）	自动摄取 + 手动维护的 Markdown 知识	修订历史 + 知识图谱

3. 典型场景

场景	涉及模块
客服智能问答：用户提问 → Agent 自动检索 FAQ / 文档库 → 给出引用了原文的回答	KB + Agent + Session
招投标文档审查：上传招标文件 → Agent 调四维度审查 + 招投标对比 → 输出修订后 docx	KB + Agent + Skills + Sandbox
投标文件编写：分析招标文件评分项 → 生成应答 → 转 docx	KB + Agent + Skills
跨部门 Wiki 沉淀：把分散的文档自动蒸馏成结构化、可搜索的知识库	KB (Wiki 模式) + Agent
多步骤业务流程自动化：客服工单 → 调 LLM 分类 → 查 KB → 调 HTTP → 写回数据库	Workflow
企业 IM 接入：把 KB 与 Agent 接入飞书 / 企微 / Slack，对员工直接对话	KB + Agent + IM
网站嵌入：在您的产品网站嵌入聊天 widget，提供基于私有 KB 的客服	KB + Agent + Embed Channel

4. 部署形态

LuoSA 通过 `docker-compose.yml` 一键拉起完整基础设施：



部署脚本详见 `AGENTS.md` 与 `docs/poc/deployment/`。

5. 登录入口与首屏

- **URL**：`https://your-domain/`
- **首次启动**（Lite 模式）：会自动初始化首个 Owner 账户，请按页面提示操作。
- **登录后默认页**：`/platform/creatChat`（新建对话首页），显示推荐问题与可用 KB。

主界面五大区域：

区域	说明
顶部导航	工作区切换、用户菜单、设置入口、邀请角标
左侧栏	KB / Agent / Workflow / 组织 / 集成的导航入口
主内容	当前选中模块的工作台
右下	全局命令面板（`Ctrl+K`）
设置模态	统一入口，通过 `?section=` 参数切换所有配置

---

6. 关键概念（先认识这些）

概念	解释
工作区（Tenant）	LuoSA 的资源隔离单位。一个用户在多个工作区可有不同角色
角色（Role）	Viewer / Contributor / Admin / Owner 四级，决定可执行的操作
平台超管（System Admin）	跨工作区的平台级运维标志
组织（Organization）	跨工作区的 KB 共享载体，使用独立的 OrgMemberRole
知识库（KB）	一组文档 + FAQ + Wiki 页面的集合，是检索的最小单元
智能体（Agent）	一个可对话的 AI 实体，绑定 KB + 工具 + 系统提示
工作流（Workflow）	一个可视化的节点 DAG，按发布版本执行
会话（Session）	一组用户消息 + Agent 回复的对话线程
API Key	程序化访问的鉴权主体，携带 capability 列表

7. 与角色相关的能力边界

> **本节是最重要的「阅读指南」**——请直接跳到对应角色的章节：

- **您是业务人员？** → roles/01-viewer.md
- **您要建立自己部门的 KB？** → roles/02-contributor.md
- **您负责公司平台 / IT？** → roles/03-admin.md
- **您是部门主管 / 团队负责人？** → roles/04-owner.md
- **您是平台 SRE / 运维？** → roles/05-system-admin.md

---

8. 阅读建议

1. 第一次使用：先读 02 快速上手 把主界面跑通。
2. 理解权限：再看 README § 角色矩阵 确认您所在的角色。
3. 按角色阅读：roles/ 下选择您对应的角色文件。
4. 深入功能：需要时查阅 modules/ 下的具体模块文档。

02 • 快速上手

> **目标**：5 分钟内从登录到完成第一次有意义的对话。所有角色都应能跟完。

1. 登录

1.1 普通登录

1. 打开浏览器访问 LuoSA 部署 URL（部署完成后由管理员提供，例如 `https://luosa.your-company.com`）。
2. 在登录页输入**用户名** + **密码**，点击「登录」。
3. 首次登录后默认进入 `/platform/creatChat`（新建对话首页）。

1.2 通过邀请链接登录

1. Owner 通过 roles/04-owner.md 创建 share-link。
2. 您收到形如 `https://luosa.your-company.com/join?code=xxx` 的链接。
3. 点击链接 → 注册页（自动填入邀请码）→ 设置密码 → 加入工作区。
4. **注意**：通过公开 `/auth/register` 自注册的用户默认未审批，必须 Admin+ 批准后才能登录（详见 roles/03-admin.md § 2 审批流程）。

1.3 OIDC / SSO 登录

如果管理员配置了 OIDC（详见 modules/06-system-settings.md § 6 OIDC），登录页会显示「以 OIDC 登录」按钮。OIDC 用户**免审批**。

2. 主界面速览

登录后看到的是这样的结构：



元素	用途
顶部 Logo	点击回到新建对话首页
工作区切换	切换到您有成员资格的其他工作区（每个工作区有独立的角色）
⌘K 命令面板	全局搜索 KB / Action / 跳转
邀请角标	您收到的待接受邀请
用户菜单	个人资料、设置、切换工作区、登出
左侧导航	模块入口（KB / Agent / 工作流 / 组织 / 设置）
主内容	当前模块的工作台

---

3. 与默认智能体对话（3 步）

> 这是所有用户（包括 Viewer）都能做的事。

第 1 步：进入新建对话首页

- 路径：`/platform/creatChat`（或点击顶部 Logo）
- 页面会显示**推荐问题卡片**（按内置 FAQ 标记）

第 2 步：选择 Agent（可选）

- 对话页顶部有一个 **Agent 选择器**（`AgentSelector.vue`）
- 第一次使用，默认是 **`builtin-quick-answer`**（直接对话，不依赖 KB）

- 如果您想测试某个 KB 的检索，可以换一个绑定了 KB 的 Agent

第 3 步：提问

- 在底部输入框输入问题，按 Enter 发送
- 流式输出回答（您能看到 LLM 一个个 token 出来）
- 如果 Agent 调用了 KB，会显示**引用 popover**（点击引用跳到原文片段）
- 点击右上角「参考资料」按钮可以查看完整引用列表

试一试：

```
提问："你好，你能做什么？"
```

默认 Agent 会自我介绍它的能力。如果工作区里有任何 KB，您可以问：

```
提问："[KB 里某篇文档标题] 的核心结论是什么？"
```

Agent 会自动检索 KB，给出带引用的回答。

---

4. 浏览一个知识库

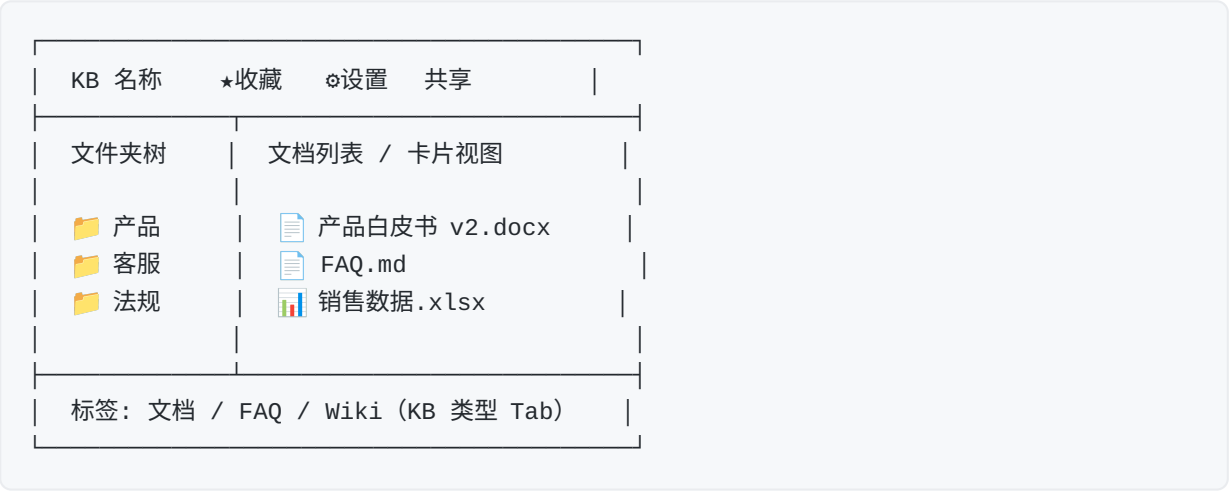
> Viewer / Contributor / Admin / Owner 都能看 KB（只读 vs 编辑权限不同）。

第 1 步：进入知识库列表

- 路径：`/platform/knowledge-bases`
- 左侧有分组：**全部 / 我的 / 收藏 / 最近 / 各组织共享**

第 2 步：选择一个 KB

- 点击任意 KB 卡片，进入 KB 详情页（`/platform/knowledge-bases/:kbId`）
- KB 详情页布局：



第 3 步：检索测试

- 顶部「试一试」按钮会跳转到带 KB 上下文的对话
- 您可以直接提问，Agent 会限定在这个 KB 里检索

---

5. 看一个智能体

> Viewer / Contributor 都能跑 Agent（受 `runnable_by_viewer` 控制）。

第 1 步：进入智能体列表

- 路径：`/platform/agents`
- 左侧分组：**全部 / 我的 / 收藏 / 最近 / 组织 / chat / workflow**（按 `display_group` 分 Tab）

第 2 步：选择一个 Agent

- **内置智能体**（所有工作区都有）：
  - `builtin-quick-answer`：默认对话
  - `builtin-doc-review-proofread-chat`：文档与招投标审查
  - `builtin-doc-generation-chat`：投标文件编写

第 3 步：点击 Agent 卡片 → 与它对话

- Agent 详情页会展示：描述、绑定的 KB、可用工具、对话按钮

6. 邀请收件箱

> 所有用户都能接收邀请。

- 顶部铃铛显示待接受邀请数量
- 点击进入邀请列表（`/platform/settings?section=myinvitations`）
- 操作：接受 / 拒绝 / 查看详情（来自哪个工作区、什么角色）

---

7. 个人资料

- 路径：`/platform/settings?section=userprofile`
- 可改：头像、显示名、密码、UI 偏好
- **修改密码要求**：8-32 字符、含字母与数字（`ValidatePassword`）
- 复杂密码开关（`complexPasswordEnabled`）可能要求更严格

---

8. 5 分钟清单（跟完即上手）

- [ ] 已登录，看到主界面
- [ ] 已与默认 Agent 对话一次
- [ ] 已浏览一个 KB，能看到文档列表
- [ ] 已看一个 Agent 的详情
- [ ] 已知道设置在哪里、邀请角标在哪里
- [ ] 已读 README 的角色矩阵 → 知道自己是 Viewer / Contributor / Admin / Owner 中的哪个

9. 下一步

按您的角色继续：

- **Viewer**：roles/01-viewer.md — 您能用平台做什么
- **Contributor**：roles/02-contributor.md — 创建第一个 KB 与 Agent
- **Admin**：roles/03-admin.md — 接入 LLM、配置基础设施
- **Owner**：roles/04-owner.md — 邀请团队成员
- **System Admin**：roles/05-system-admin.md — 平台级运维

模块参考（按需查阅）：

- modules/01-knowledge-base.md
- modules/02-agent.md
- modules/03-workflow.md
- modules/04-session-chat.md
- modules/05-skills-mcp.md
- modules/06-system-settings.md
- modules/07-wiki.md

角色手册 • Viewer（访客 / 业务人员）

> **对应后端角色**：`tenant_members.role = 'viewer'`（level 10） **典型用户**：业务分析师、客服、市场调研、销售支持、新员工（只读访问部门资料） **核心心智**：**只读 + 对话 + 跑公开 Agent**。您不需要懂技术，会用浏览器提问就够了。

---

1. 您的能力地图

✅ 您能做的事

操作	入口	说明
浏览 / 检索所有 KB	`/platform/knowledge-bases`	看文档列表、用 KB 试问
浏览所有 Agent	`/platform/agents`	看 Agent 详情、描述、绑定 KB
与任何已发布的 Agent 对话	`/platform/chat/:chatid`	流式输出、引用 popover
跑 `runnable_by_viewer=true` 的 Agent	同上	关键判断字段
查看工作区成员名单	`/platform/settings?section=members`	只读
接收邀请 / 主动加入其他工作区	邀请收件箱 / `/join?code=xxx`	—
修改自己的个人资料	`/platform/settings?section=userprofile`	头像 / 密码 / 偏好
创建会话、保存对话历史	`/platform/chat/:chatid`	创建会话需要 `Contributor+`（`g.Contributor()`）

> ⚠️ 一些 UI 操作（如「新建对话」）会要求 Contributor 角色。若您看到「403」或按钮隐藏，请联系管理员调整角色。

❌您不能做的事

操作	原因 / 谁能做
创建知识库	Contributor+（`g.Contributor()`）
创建智能体	Contributor+（`g.Contributor()`）
创建工作流	Contributor+（`g.Contributor()`）
上传文档到 KB	Contributor+ 且为 KB 的创建者（`g.OwnedKBOrAdmin()`）
编辑任何 KB / Agent 配置	Contributor 仅自己名下；Admin+ 可编辑所有
配置基础设施（模型、向量库、MCP 等）	Admin+
邀请成员 / 修改成员角色	Owner+
删除空间	Owner+

---

2. 典型工作流

场景 A：检索一个已有 KB

> **前置**：管理员已经创建并填充了一个 KB，例如「2025 销售手册」。

1. 进入 **知识库列表** `/platform/knowledge-bases`
2. 找到「2025 销售手册」，点击进入
3. 切换**文档列表 / 卡片**视图，按文件夹浏览或搜索文件名
4. 点击 KB 顶部「**试一试**」按钮 → 跳转到带 KB 上下文的对话
5. 提问，例如：「2025 年 Q3 销售额是多少？」
6. 在回答中点击引用 popover，**跳到原文位置**

场景 B：与 Agent 对话

1. 进入 **Agent 列表** `/platform/agents`
2. 找一个您感兴趣的 Agent（例如「客服 FAQ Bot」），点击进入
3. 点击「**开始对话**」按钮
4. 提问，回答会带引用
5. 可以**固定上下文**（pin）这条会话，反复追问

场景 C：接受一个邀请

1. 收到邀请邮件 / 链接 `https://.../join?code=xxx`
2. 点击链接，进入「**注册 / 接受**」页
3. 设置密码（如果还没账号），确认接受
4. 邀请方收到通知，您以指定角色加入工作区

场景 D：查看工作区成员

1. 进入 **设置** `/platform/settings?section=members`
2. 浏览成员名单（**只读**）
3. 如果需要联系 Owner，使用头像/邮箱

---

3. 关键页面详解

3.1 对话主界面（`/platform/chat/:chatid`）

区域	内容
顶部	Agent 选择器、会话标题、参考资料按钮、设置菜单
主体	用户消息 / 助手消息（Markdown）、引用 popover、follow-up 建议
底部	输入框、上传附件、模型设置、IM 来源标识

关注点：

- **引用 popover**：每条回答下方的「[1] [2]」小标号，hover 看原文片段
- **参考资料抽屉**：查看完整引用列表（KB 名称、文档名、chunk ID）
- **附件**：可以上传临时文件（图片 / 文档），Agent 会自动读取
- **follow-up 建议**：回答末尾的「推荐下一步问题」

3.2 知识库详情（`/platform/knowledge-bases/:kbId`）

区域	内容
顶部	KB 名、收藏、设置（只读）、共享
左侧	文件夹树 + 文档列表 / 卡片切换 + FAQ Tab + Wiki Tab
主体	文档列表（解析状态、tags、来源、上传时间）

只读用户可见的元素：

- 文档预览（hover 看摘要）
- 解析状态时间轴（pending / parsing / done / failed）
- 标签 / FAQ 条目
- Wiki 浏览、修订历史查看（**只读**，不能编辑）

3.3 Agent 列表（`/platform/agents`）

区域	内容
Tab	全部 / chat / workflow（按 `display_group` 分类）
左侧	全部 / 我的 / 收藏 / 最近 / 组织
卡片	Agent 名、描述、display_group、绑定的 KB 数、运行按钮

---

4. 常见问题（FAQ）

Q1：为什么我看不到「新建对话」按钮？

A：创建会话需要 Contributor+ 角色。请联系工作区 Owner 提升您的角色。

Q2：为什么我能跑某个 Agent 但跑不了另一个？

A：Agent 有一个 `runnable_by_viewer` 标记。Viewer 只能跑标记为 true 的 Agent。如果该 Agent 是您部门常用的，请联系创建者开启标记或提升您的角色。

Q3：我能下载 Agent 引用的原文吗？

A：可以。在对话的「参考资料」抽屉中，每条引用都有 KB 名称、文档名、链接。点击 KB 名称 → 文档详情 → 下载原始文件（如 KB 启用了分享）。

Q4：为什么我看到的是「待审批」状态？

A：自注册的用户默认未审批。请联系工作区 Admin / Owner 批准（详见 roles/03-admin.md § 2）。

Q5：能在移动端用吗？

A：可以。Web 控制台是响应式设计，但建议在桌面浏览器使用。移动端原生体验通过 IM 集成（飞书 / 企微 / Slack）更优。

5. 何时联系 Owner

如果出现以下情况，请联系您的 Owner：

情况	谁该处理
需要创建 KB / Agent	Owner 提升您为 Contributor，或 Owner 自己创建
需要修改 KB 内容	Contributor 创建者，或 Admin
需要接入新的 LLM / 工具	Admin
邀请新成员	Owner
工作区审计问题	Admin / Owner
系统级问题（登录失败、KB 不可访问）	Admin / Owner

6. 进一步阅读

- modules/04-session-chat.md — 会话与聊天完整功能
- modules/01-knowledge-base.md § 4 — 知识库浏览与检索
- modules/02-agent.md § 3 — 智能体使用
- README.md § 角色矩阵 — 确认您的权限边界

角色手册 • Contributor（编辑）

> **对应后端角色**：`tenant_members.role = 'contributor'`（level 20） **典型用户**：领域专家、产品经理、运营、HR（写自己负责的 KB）、写智能体的业务人员 **核心心智**：**您是「创作者」**。能创建 KB / Agent / 上传文档 / 调试 Agent，但不能动基础设施和团队成员。

1. 您的能力地图

✅ 您能做的事

操作	中间件门	说明
创建知识库（自己名下）	`g.Contributor()`	KB 类型：Document / FAQ / Wiki
创建智能体（自己名下）	`g.Contributor()`	自定义 Agent
创建工作流（自己名下）	`g.Contributor()`	Dify-parity 节点编排
创建会话	`g.Contributor()`	`/platform/creatChat`
上传文档 / URL / 文件夹到自己 KB	`g.OwnedKBOrAdmin()`	触发解析任务
编辑自己创建的 KB / Agent / Workflow	`g.OwnedKBOrAdmin()`	自己是 `creator_id`
复制 KB / Agent（成为副本的创建者）	`CopyKnowledgeBase`	副本默认名「原名 的副本」
启用 / 关闭 Agent 的 `runnable_by_viewer`	`g.OwnedKBOrAdmin()`	让 Viewer 也能跑
浏览 / 检索他人资源	Viewer 默认放行	只读
修改个人资料	自己	—

⚠️ 部分能做（条件性）

操作	条件
编辑他人 KB / Agent	您是该资源的 `creator_id`，**或** Admin+ 帮您提升
启用 KB 共享	KB 创建者可以启用 Owner 给出的共享策略

❌ 您不能做的事

操作	谁能做
配置基础设施（模型 / 向量库 / MCP / 沙箱 / Skill / 解析器 / 存储 / IM / 记忆 / Web 搜索）	Admin+（`g.Admin()`）
邀请 / 移除 / 修改成员	Owner+（`g.Owner()`）
修改空间配置（空间名、quota、API Key）	Owner+
批准待审批用户	Admin+（`RequireRoleOrSystemAdmin(Admin, …)`）
查看审计日志	Admin+
删除工作区	Owner+

---

2. 典型工作流

场景 A：建立您的第一个知识库

> **目标**：把「客服 FAQ」整理成可检索的知识库。

1. 进入 `/platform/knowledge-bases`，点击右上角「**+ 新建知识库**」（Contributor+ 显示）
2. 填写：
   - **名称**：例如「客服 FAQ 2025」
   - **类型**：FAQ
   - **描述**：写清楚这个 KB 的范围
   - **嵌入模型**：默认工作区配置（Admin 设置过）
   - **向量库 / 存储后端**：默认工作区配置
   - **共享策略**：默认私有
3. 点击「**创建**」→ 自动跳转到 KB 详情页
4. 进入「**FAQ**」Tab，开始添加问答对
5. 也可以在「**文档**」Tab 上传文件（FAQ / Markdown）

场景 B：上传文档到 KB

> **前置**：您是 KB 的创建者，或 KB 是您所在部门的公共 KB。

1. 进入 KB 详情页
2. 点击「**上传**」按钮（三种来源：文件 / URL / 文件夹）
3. 选择源：
   - **文件**：拖拽或点击上传（PDF / Word / Excel / 图片 / EPUB / XMind …）
   - **URL**：粘贴 URL，自动抓取
   - **文件夹**：批量上传
4. 上传后**自动触发解析**（派发到 DocReader gRPC，端口 50051）
5. 在文档列表查看解析状态（pending / parsing / done / failed）
6. 解析完成后**自动切片 + 嵌入**，可立即检索

场景 C：调 KB 解析参数

> KB 创建者可以修改解析配置（`g.OwnedKBOrAdmin()`）。

1. 在 KB 详情页右上角点击「⚙ 设置」 → 「**解析设置**」
2. 调整：
   - 解析器（默认 / MinerU / MarkItDown / OpenDataLoader）
   - 切片策略（按段落 / 固定窗口 / 语义切片）
   - 切分粒度、重叠区间
3. 「**保存**」→ 对新上传的文档生效；对已有文档可点「**重新解析**」

> **更多 KB 配置项**（13 个子面板）：详见 modules/01-knowledge-base.md § 5

场景 D：创建您的第一个 Agent

> **目标**：写一个绑定「客服 FAQ」KB 的客服 Agent。

1. 进入 `/platform/agents`，点击右上角「**+ 新建智能体**」
2. 填写：
   - **名称**：「客服 Bot」
   - **类型**：`smart-reasoning`（推荐）
   - **系统提示**：从 `config/agent_type_presets.yaml` 自动填入推荐
3. 进入「**知识库**」标签，选择「客服 FAQ 2025」

4. 进入「**工具**」标签，按需启用：
   - `knowledge_search`（KB 检索）— **默认开**
   - `grep_chunks`（按关键词搜索块）
   - 其他工具按需
5. 进入「**对话策略**」标签，调整 temperature / max_tokens
6. 点击「**保存**」
7. 点击右上角「**测试**」按钮，在右侧对话面板测试 Agent

场景 E：把 Agent 标记为 Viewer 可跑

1. 在 Agent 详情页找到「**允许 Viewer 运行**」开关（`runnable_by_viewer`）
2. 打开后，Viewer 角色也能跑这个 Agent

场景 F：复制别人的 KB / Agent

1. 在 KB / Agent 列表，找到「⋮」菜单
2. 点击「**复制**」→ 生成「原名 的副本」
3. 您成为副本的创建者，可编辑

---

3. 关键页面详解

3.1 KB 列表（`/platform/knowledge-bases`）

操作	说明
「+ 新建知识库」	Contributor+ 可见
「⋮ 复制」	创建副本，您成为创建者
「⋮ 启用共享」	Owner 配置共享策略后，您可启用
「⋮ 收藏」	收藏到左侧「我的」

3.2 KB 详情（`/platform/knowledge-base/:kbId`）— 您作为创建者

Tab	操作
文档	上传、批量打标签、移动文件夹、删除（`OwnedKBOrAdmin`）
FAQ	增删改条目（`OwnedKBOrAdmin`）
Wiki	浏览 + 编辑 Markdown（`OwnedKBOrAdmin`），查看修订历史、回滚
设置	13 个子面板（解析 / 切片 / 向量库 / 存储 / 模型 / 高级 / 图谱 / 共享 / 活动审计 / 数据源）

3.3 Agent 编辑器（`/platform/agents` 详情）

左侧导航分组：

分组	用途
基础	名称、描述、type、display_group、tags
提示词	system prompt + welcome message + 推荐问题
模型	temperature / max_tokens / 模型选择
知识库	选择 KB + 选择具体文档 + capability 过滤
工具	知识检索 / 文件操作 / 数据 / MCP / Skill / 多模态
技能	启用 Skill 列表（`SkillsEnabled` + `SkillsSelectionMode`）
沙箱	选择沙箱后端（Docker / E2B / Cube）
对话策略	temperature / max_tokens / follow-up
问题建议	推荐问题管理

4. 创建资源的「标准流程」清单

步骤	KB	Agent	Workflow
1. 进入模块入口	`/platform/knowledge-bases`	`/platform/agents`	`/platform/workflow`
2. 点新建	「+ 新建知识库」	「+ 新建智能体」	「+ 新建工作流」
3. 填元信息	名称 / 类型 / 描述	名称 / type / 描述	名称 / 描述
4. 配置核心	解析 / 切片 / 向量库	提示词 / 工具 / KB / 沙箱	节点编排 / 变量 / DSL
5. 保存	保存	保存	保存（自动保存）
6. 测试	上传文档 → 试检索	「测试」按钮 → 对话	「运行」按钮 → 看节点输出
7. 发布（Agent / Workflow）	标记 `runnable_by_viewer`（可选）	「发布」	「发布」（冻结 DSL）

5. 常见问题（FAQ）

Q1：为什么我编辑不了别人的 KB?

A：您只对自己创建的 KB 有写权限（`g.OwnedKBOrAdmin()`）。请 Owner 联系 Admin 把您提升角色，或者复制那个 KB 改成自己的。

Q2：文档上传后一直「解析中」?

A：可能 DocReader 服务异常。请让 Admin 检查：

- DocReader gRPC 服务是否启动（端口 50051）
- Redis / Postgres 连接是否正常
- 在 `docker logs WeKnora-docreader` 看解析日志 详见 modules/01-knowledge-base.md § 4 排错。

Q3：我能修改工作区的全局模型配置吗？

A：不能。模型配置是 **Admin 限定**（`g.Admin()`）。请联系 Admin。

Q4：为什么我新建 Agent 时某些工具看不到？

A：某些工具受 `agent_type_presets.yaml` 的 allowlist 控制。换 `agent_type` 会自动切换工具集。

Q5：怎么把 Agent / KB 共享给其他工作区？

A：跨工作区共享由 Owner 启用后，您才能看到共享设置。详见 roles/04-owner.md § 4。

---

6. 何时升级到 Admin

如果您的工作越来越大，建议联系 Owner 升级到 **Admin**：

场景	需要 Admin
您开始负责整个部门的 KB 治理	✅
您要接入新的 LLM / 向量库 / IM 通道	✅
您要处理待审批用户	✅
您要管理 MCP 服务 / Skill 安装	✅
您需要查看审计日志	✅

---

7. 进一步阅读

- modules/01-knowledge-base.md — 知识库完整功能
- modules/02-agent.md — 智能体完整功能
- modules/03-workflow.md — 工作流完整功能
- modules/04-session-chat.md — 会话与测试
- modules/07-wiki.md — Wiki 模式编辑
- README.md § 角色矩阵

角色手册 • Admin（租户管理员）

> **对应后端角色**：`tenant_members.role = 'admin'`（level 30） **典型用户**：IT 运维、平台管理员（公司内部）、算法工程师、DevOps **核心心智**：**您是「基础设施运维者」**。配置模型 / 向量库 / MCP / IM / Skill / 沙箱 / 解析器 / 存储，批准待审批用户，查看审计，但不能邀请成员或改空间配置。

---

1. 您的能力地图

✅ 您能做的事

> **Admin 是所有 Contributor 能力的超集**，加上：

大类	操作
模型管理	创建 / 更新 / 删除 LLM、Embedding、Rerank、VLM、ASR 模型；测试连接
向量库	配置 Qdrant / pgvector / Weaviate / Milvus 等
存储后端	配置 MinIO / S3 / 腾讯 COS / 阿里 OSS
解析器	选择 DocReader 解析模式、MinerU 端点
MCP 服务	创建 / 测试 / 配置 OAuth / 工具列表
Skill 安装	从 ClawHub / SkillHub / Git / Zip 安装工作区 Skill
沙箱	配置 Docker / E2B / Cube 后端、网络策略、TTL
网络搜索	配置 Web 搜索引擎（Bing / Tavily / Google …）
长期记忆	配置 Memory 抽取策略、用户确认流程
聊天历史	配置会话保留策略
IM 集成	配置飞书 / 企微 / Slack / Telegram / Lark / 钉钉 / QQBot / Mattermost / 微信 / 云之家
Embed 通道	配置网站嵌入 widget
待审批用户	批准 / 拒绝待审批用户（`RequireRoleOrSystemAdmin(Admin, …)`）
审计日志	查看工作区审计日志（`Admin()` 门）
评估	跑评估数据集、查看 BLEU / ROUGE / Recall hit rate
任务队列	查看失败任务、DLQ、worker pool 状态（与 System Admin 部分协作）
环境变量	设置工作区 Env Var
存储配额	应用默认存储配额（`ApplyDefaultStorageQuota`，跨租户超管也有）

⚠️ 部分能做（条件性）

操作	条件
编辑他人 KB / Agent / Workflow	`g.OwnedKBOrAdmin()` 直接覆盖

❌ 您不能做的事

操作	谁能做
邀请 / 移除 / 修改成员	Owner+（`g.Owner()`）
修改空间配置（空间名、quota、`is_joinable`、API Key）	Owner+
删除工作区 / 转让所有权	Owner+
提升 / 撤销系统管理员	仅 System Admin
平台 API Key / 平台系统设置	仅 System Admin

---

2. 待审批用户流程（高频操作）

> 自注册用户默认 `is_approved=false`。Admin+ 在「待审批」中处理。

第 1 步：进入待审批队列

- 路径：`/platform/settings?section=pending-users`（Admin+ 可见）
- 或 `frontend/src/views/system/PendingUsers.vue`

第 2 步：审查申请

每条申请显示：

字段	说明
用户名	注册时填写
邮箱	联系用
注册时间	时间线
申请附言（可选）	用户填写的理由

第 3 步：批准 / 拒绝

- 点击「**批准**」→ 用户登录状态解锁，自动获得默认角色（contributor）
- 点击「**拒绝**」→ 用户被禁用

> 后端 API：
>
> - `GET /api/v1/system/admin/pending-users`
> - `POST /api/v1/system/admin/users/:id/approve`
> - `POST /api/v1/system/admin/users/:id/reject`

---

3. 配置基础设施的标准流程

场景 A：接入新的 LLM（OpenAI / DeepSeek / Claude / 国产模型…）

1. 进入 `/platform/settings?section=models`
2. 点击「**+ 新建模型**」
3. 选择：
   - **提供商**：OpenAI / Azure OpenAI / Anthropic / DeepSeek / Qwen / Zhipu / Hunyuan / Doubao / Gemini / MiniMax / NVIDIA / Novita / SiliconFlow / OpenRouter / Requesty / LiteLLM / Ollama（`BUILTIN_MODELS.md`）
   - **类型**：Chat / Embedding / Rerank / VLM / ASR
4. 填入凭证（API Key / endpoint / model name）
5. 点击「**测试连接**」→ 验证可用
6. 保存 → 选择**作用范围**：
   - 全工作区默认
   - 特定 KB 覆盖
   - 特定 Agent / Workflow 覆盖
7. 在「使用详情」中查看消耗

> 详细模型清单见 `config/builtin_models.yaml` + `docs/BUILTIN_MODELS.md`。

场景 B：配置向量库

1. `/platform/settings?section=vectorstore`
2. 选择后端：pgvector（默认）/ Qdrant / Milvus / Weaviate
3. 填入连接信息
4. 测试 → 保存
5. KB 创建时默认使用工作区配置；可逐 KB 覆盖（详见 modules/06 § 5）

场景 C：添加 MCP 服务

1. `/platform/settings?section=mcp`
2. 「**+ 新建 MCP 服务**」→ 选择：
   - **传输协议**：stdio / SSE / HTTP
   - **认证**：None / API Key / OAuth2
3. 填入端点 / 凭证
4. 点击「**测试连接**」→ 拉取工具列表
5. 保存 → 在 Agent 编辑器中可用

场景 D：安装工作区 Skill

1. `/platform/settings?section=skills`
2. 「**+ 安装技能**」→ 选择源：
   - **ClawHub**：官方市场
   - **SkillHub**：社区市场
   - **Git**：直接 Git URL
   - **Zip**：上传 zip 包
3. 选择安装路径（工作区级 vs 个人级）
4. 安装过程显示进度 → 完成
5. 在 Agent 编辑器中可用（`SkillsEnabled` + `SkillsSelectionMode`）

场景 E：配置沙箱

1. `/platform/settings?section=sandbox`
2. 选择后端：**Docker**（默认 / 集群）/ **E2B** / **Cube**
3. 填入 Docker socket / E2B API Key / Cube endpoint
4. 设置网络策略（默认限制出网）
5. 设置 TTL / 资源限制（CPU / 内存）
6. 保存 → Agent 调用 `shell_exec` / Skill 脚本时使用

场景 F：配置 IM 通道

1. `/platform/settings?section=integrations`
2. 选择平台：WeCom / Feishu / Lark（国际版）/ QQBot / Slack / Telegram / DingTalk / Mattermost / WeChat / Yunzhijia
3. 按各平台指南填入 App ID / Secret / Webhook
4. 测试 → 启用

5. 用户在 IM 中可直接与 Agent 对话

> IM 集成开发细节详见 `docs/IM集成开发文档.md`。

场景 G：配置数据源

> 把外部系统（飞书 Wiki / GitLab / Notion / Yuque / RSS / 腾讯 IMA 等）的文档自动同步到 KB。

1. KB 详情 → 设置 → 数据源
2. 「**+ 添加数据源**」
3. 选择 connector（Feishu Wiki / Feishu Drive / Lark / GitLab / 腾讯 IMA / Notion / Yuque / RSS）
4. 填入凭证 + 同步路径
5. 「**测试**」→ 「**启用**」→ 周期同步（可看同步日志）

---

4. 审计日志

- 路径：`/platform/settings?section=audit-log`（Admin+）
- 记录：
  - 成员变更（邀请、接受、拒绝、改角色、移除）
  - RBAC 拒绝事件（1 分钟滑动窗口去重）
  - 关键资源变更（KB 删除、Agent 发布、API Key 创建）
- 用途：合规审计、安全调查

---

5. 评估（Evaluation）

- 设置页 → 评估
- 创建评估数据集（QA 对 + 期望答案）
- 跑端到端评估：BLEU / ROUGE / Recall hit rate
- 对比不同 KB / 模型 / Agent 配置的效果

---

6. 任务队列与 Worker Pool 治理

- 与 System Admin 协作

- Admin 可查看：失败任务、DLQ、重试
- 详细运行时队列治理（worker pool / per-model 并发）：详见 roles/05-system-admin.md § 5

---

7. 关键页面详解

7.1 设置模态（统一入口）

`/platform/settings` 通过 `?section=` 切换。所有 section + 最低角色：

section	最低角色	说明
`general`	viewer	通用偏好
`models`	**admin**	LLM / Embedding / Rerank / VLM
`ollama`	admin	本地 Ollama
`websearch`	admin	Web 搜索
`chathistory`	admin	聊天历史
`vectorstore`	admin	向量库
`parser`	admin	文档解析引擎
`storage`	admin	存储后端
`sandbox`	admin	沙箱
`skills`	admin	Skills 工作区安装
`mcp`	admin	MCP 服务
`memory`	admin	长期记忆
`tenant`	viewer	工作区信息（**Owner 才能编辑**）
`members`	viewer（快捷入口 owner）	成员管理（**Owner 才能操作**）
`userprofile`	viewer	个人资料
`envvars`	viewer	环境变量
`pending-users`	**admin**	待审批用户
`audit-log`	admin	审计日志
`integrations`	viewer	集成（API Key Playground + IM 落地页）

7.2 System Admin 专属 section（Admin 看不到）

section	谁能看
`system-global`	System Admin
`runtime-queues`	System Admin
`platform-api-keys`	System Admin
`system-audit-log`	System Admin

> 来源：`frontend/src/config/settingsAccess.ts`

---

8. 常见问题（FAQ）

Q1：为什么我点不动「成员」管理?

A：成员管理是 **Owner 限定**（`g.Owner()`）。请联系 Owner 处理。

Q2：为什么我看不到「平台 API Key」入口?

A：平台 API Key 仅 System Admin 可见。工作区 API Key 在 Owner 视角下。

Q3：我能调整 KB 的解析参数吗?

A：可以，作为 KB 创建者直接调（`OwnedKBOrAdmin`）；或者作为 Admin 修改工作区默认解析引擎。

Q4：审批完用户后，他还需要做什么?

A：不需要。批准后用户状态解锁，下一次登录即可进入工作区。

Q5：MCP 服务的 OAuth2 中途授权怎么处理?

A：在 MCP 设置里配置 OAuth2 后，Agent 在对话中调用需要授权的工具时，会自动触发 OAuth flow（mid-conversation），用户会看到授权弹窗。

---

9. 何时升级到 Owner

如果您负责整个工作区的治理（不仅是基础设施），请联系当前 Owner 升级：

场景	需要 Owner
您要管理团队成员	✅
您要修改空间名 / 配额	✅
您要管理租户 API Key	✅
您要删除 / 归档空间	✅

10. 进一步阅读

- modules/05-skills-mcp.md — Skills 与 MCP 完整功能
- modules/06-system-settings.md — 系统设置完整索引
- roles/05-system-admin.md — 平台级运维（如果您负责多租户）
- `docs/BUILTIN_MODELS.md` — 内置模型清单
- `docs/IM集成开发文档.md` — IM 集成完整文档
- `docs/数据源导入开发文档.md` — 数据源开发文档
- README.md § 角色矩阵

角色手册 • Owner（所有者）

> **对应后端角色**：`tenant_members.role = 'owner'`（level 40） **典型用户**：团队负责人、部门主管、空间创建者、CTO **核心心智**：**您是「空间负责人」**。所有 Admin 的能力 + 邀请/移除成员 + 修改空间配置 + 管理 API Key + 删除空间。

---

1. 您的能力地图

✅ 您能做的事

> **Owner 是所有 Admin 能力的超集**，加上：

大类	操作
成员管理	邀请 / 移除 / 修改角色 / 主动添加 / 退出（不能退出最后一位 Owner）
邀请流程	创建 share-link（`/auth/invite-links`）、邀请已注册用户、撤销邀请
空间配置	修改空间名 / 描述 / `is_joinable` / `storage_quota` / 业务标识
租户 API Key	创建 / 列出 / 撤销（携带 capability + KB allow-list）
删除空间	异步清理（`ProcessKBDelete`），不可恢复
转让所有权	转让给其他成员（把自己降级 + 把对方升级）
审计日志	同 Admin，可看完整日志
跨工作区	创建 Organization，把 KB 共享给组织

❌ 您不能做的事

操作	谁能做
提升 / 撤销系统管理员	仅 System Admin
平台 API Key / 平台系统设置 / 运行时队列	仅 System Admin

2. 邀请新成员

场景 A：邀请一个已注册用户

1. 进入 `/platform/settings?section=members`
2. 点击「**+ 邀请成员**」
3. 填入：
   - **邮箱 / 用户名**（必须已注册）
   - **角色**：viewer / contributor / admin / **owner**（谨慎）
   - **附言**（可选）
4. 提交 → 状态 `pending` → 收件箱里对方看到邀请
5. 对方接受后，状态 `accepted`，写入 `tenant_members`

> 后端 API：
>
> - `POST /api/v1/tenants/:id/invitations`

场景 B：创建 share-link（让对方自服务注册）

1. 进入 `/platform/settings?section=members` →「**分享链接**」Tab
2. 点击「**+ 创建链接**」
3. 设置：
   - **角色**：viewer / contributor / admin
   - **有效期**：默认 7 天（`RBAC_INVITATION_TTL`）
   - **使用次数**：单次 / 多次
4. 复制链接：`https://.../join?code=xxx`
5. 发给候选人 → 对方点击 → 注册 → 自动加入（**绕过待审批**）

> 后端 API：
>
> - `POST /api/v1/tenants/:id/invite-links`

场景 C：撤销未接受的邀请

1. 进入「邀请」列表
2. 找到 `pending` 状态的邀请

3. 点击「**撤销**」→ 状态 `revoked`，链接失效

---

3. 修改成员角色

1. 进入「成员」列表
2. 找到目标成员 → 点击「⋮」→ 「**修改角色**」
3. 选择新角色 → 确认

注意：

- ⚠️ **最后一位 Owner 不可降级**：系统返回 `409 + ErrLastOwner`
- 如果要换 Owner，先把对方提升为 Owner，再把自己降级
- Owner 转让后，对方成为新 Owner，原 Owner 可以是 Admin / Contributor / Viewer

> 后端 API：
>
> - `PUT /api/v1/tenants/:id/members/:user_id`

---

4. 移除成员

1. 进入「成员」列表
2. 找到目标成员 → 点击「⋮」→ 「**移除**」
3. 确认 → 状态变 `removed`（软删除 `deleted_at`）

限制：

- 不能移除最后一位 active Owner

> 后端 API：
>
> - `DELETE /api/v1/tenants/:id/members/:user_id`

---

5. 修改空间配置

1. 进入 `/platform/settings?section=tenant`

2. 可编辑：
   - **空间名**（`TenantInfo.vue` 中 `canEditTenant = hasRole('owner')`）
   - **描述**
   - **`is_joinable`**：是否允许其他工作区成员直接 join（Organization 共享场景）
   - **`storage_quota`**：存储配额
   - **`business`**：业务标识（用于多工作区区分）
3. 保存

> 后端 API：通过 `/tenants/:id` PUT

---

6. 租户 API Key 管理

> 用于程序化访问（机器人 / 第三方系统 / CI）。

1. 进入 `/platform/settings?section=integrations` → 「**API Keys**」Tab
2. 「**+ 创建 API Key**」
3. 配置：
   - **名称**：易记的描述
   - **能力列表**（`APIKeyCapability`，勾选）：
     - `retrieve`：检索 KB
     - `chat`：聊天
     - `ingest`：上传文档
     - `manage_kbs`：管理 KB
     - `manage_agents`：管理 Agent
     - `run_workflows`：跑工作流
     - `manage_models`：管模型
     - `manage_members`：管成员
     - `system_*`：系统级（System Admin 限定）
   - **KB allow-list**：限制该 Key 只能访问哪些 KB（强安全）
   - **过期时间**：可设置
4. 生成后**立即复制 Key**（明文只显示一次）

> API Key 与 JWT 用户是**正交**的两个鉴权维度。Key 默认拒绝未在 `apiKeyRouteAuthorizer` 注册的路由。

---

7. 跨工作区共享（Organization）

> 把 KB 共享给跨部门的团队。

1. 进入 `/platform/organizations`
2. 「**+ 新建组织**」→ 填名称 → 保存
3. 进入组织详情 → 「**成员**」 → 添加其他工作区的成员（按邮箱）
4. 设置成员的 `OrgMemberRole`：
   - `admin`：完全控制共享 KB
   - `editor`：编辑共享 KB
   - `viewer`：只读
5. 在 KB 详情 → 「**共享**」Tab → 启用共享给哪些组织

> 跨工作区 = **第二层 RBAC**，与租户角色独立。

---

8. 转让所有权

> 把 Owner 身份转给其他成员。

1. **第一步**：先把目标成员提升为 Owner（详见 § 3）
2. **第二步**：把自己的角色降级（这时对方是 Owner，您是 Admin）
3. **第三步**（可选）：自己退出空间

或者使用转让专用的「**转让所有权**」功能（如果有 UI），一步完成。

---

9. 删除空间

> ⚠️ **不可恢复**。会异步清理所有 KB、Agent、Workflow、文档、Wiki。

1. 进入 `/platform/settings?section=tenant`

2. 滚动到底部 → 「**删除空间**」
3. 输入空间名确认
4. 提交 → 异步任务 `ProcessKBDelete` 启动
5. 几分钟后空间从列表消失

约束：

- 不能删除还有成员的活跃空间（要先移除所有成员）
- 不能删除您**不在**的工作区（System Admin 可）

---

10. 关键页面详解

10.1 成员管理（`/platform/settings?section=members`）

元素	说明
「+ 邀请成员」	邀请已注册用户
「+ 创建链接」	创建 share-link
「邀请」Tab	查看 / 撤销邀请
「成员」列表	角色筛选、搜索、修改角色、移除
角色权限矩阵	内嵌展示 4 级角色能做什么（来自 `TenantMembers.vue` 的 `roleMatrix`）

10.2 空间信息（`/platform/settings?section=tenant`）

字段	谁能编辑
空间名	Owner
描述	Owner
`is_joinable`	Owner
`storage_quota`	Owner
`business`	Owner
创建时间 / 配额使用	只读

11. 常见问题（FAQ）

Q1：为什么我点不动「删除空间」按钮?

A：检查是否有未移除的成员。先移除所有成员（除了您自己），再删。

Q2：share-link 有有效期吗?

A：有，默认 7 天（`RBAC_INVITATION_TTL`）。也可设置一次性 / 多次使用。

Q3：通过 share-link 加入的用户需要审批吗?

A：**不需要**。这是绕过待审批的快速通道。

Q4：API Key 泄露了怎么办?

A：立即在设置页**撤销**（revoke），重新创建。撤销是不可逆的。

Q5：我能把 Owner 转给不在本工作区的用户吗?

A：不能。Owner 必须是当前工作区的成员。先邀请对方加入 → 提升为 Owner。

Q6：Owner 能退出自己的空间吗?

A：如果您是**最后一位 Owner**，不能退出。先把 Owner 转给别人。

---

12. 工作流检查清单（季度）

- [ ] 成员名单是最新的吗？有没有已离职的遗留?
- [ ] Owner / Admin 是否过多？（建议每空间 ≤ 3 个 Owner）
- [ ] 审计日志是否有异常拒绝?
- [ ] share-link 是否还有过期的?
- [ ] API Key 是否还有未使用的?

- [ ] 存储配额使用率？（避免超限）

---

13. 进一步阅读

- modules/06-system-settings.md § 3 租户 API Key — API Key 完整说明
- modules/06-system-settings.md § 4 Organization — 跨工作区
- `docs/RBAC说明.md` — RBAC 设计原文
- README.md § 角色矩阵
- README.md § 关键不变式

角色手册 • System Admin（平台系统管理员）

> **对应后端标志**：`users.is_system_admin = true`（独立于租户角色） **配合**：`users.can_access_all_tenants` + 集群级 `enable_cross_tenant_access` **典型用户**：平台运营、SRE、安全审计、平台 CTO **核心心智**：**您是「跨租户超管」**。平台级运维 + 跨租户切换 + 平台 API Key + 运行时队列 + 平台审计。**不受 `EnableRBAC` 开关影响**——始终强制。

---

1. 您的能力地图

✅ 您能做的事

> **System Admin 与租户角色正交**。切到任意租户时被授予**临时 Admin**（不写入 `tenant_members`），不能被授予临时 Owner（删除租户、Owner-only 操作仍需真实 Owner）。

大类	操作
跨租户切换	切换到任意工作区，临时 Admin 权限
系统管理员管理	提升 / 撤销其他 System Admin、列出所有 System Admin
平台 API Key	创建 / 列出 / 删除平台作用域 API Key
平台系统设置	集群级配置（与工作区设置级 API 不同）
运行时队列	查看 worker pool、per-stage 池、per-model 并发、失败任务、DLQ
平台审计日志	跨租户审计查询
用户审批	跨租户查看待审批用户、批准 / 拒绝
用户管理	重置任意用户密码、创建用户、禁用用户
存储配额	应用默认存储配额（`ApplyDefaultStorageQuota`）
管理所有工作区	Owner / Admin 的所有能力（跨租户）

⚠️ 与 Owner 的边界

操作	您	Owner
修改本工作区成员	✅	✅
删除本工作区	⚠️ 跨租户访问需要双重开关打开	✅
转让所有权	❌（不写 `tenant_members`）	✅
Owner-only 操作	❌（您只是临时 Admin）	✅

---

2. 平台管理员入口

System Admin 专属页面都在 `/platform/system/*` 路由下（实际重定向到 `/platform/settings?section=...`）。

section	实际页面	用途
`system-global`	`system/SystemSettings.vue`	平台级系统设置
`runtime-queues`	`system/RuntimeQueues.vue`	运行时队列 + worker pool
`system-audit-log`	`system/SystemAuditLog.vue`	平台审计
`pending-users`	`system/PendingUsers.vue`	待审批用户
`platform-api-keys`	`system/PlatformAPIKeys.vue`	平台 API Key

> 路由守卫：`requiresSystemAdmin === true` → 检查 `authStore.isSystemAdmin`，否则跳走。详见 `frontend/src/router/index.ts`。

---

3. 提升 / 撤销其他 System Admin

> ⚠️ 谨慎操作。System Admin 是平台级权限。

提升

1. 进入 `/platform/system/system-global` → 「**系统管理员**」Tab
2. 输入目标用户邮箱

3. 「**提升为系统管理员**」

> 后端 API：
>
> - `POST /api/v1/system/admin/promote`

撤销

1. 同上
2. 找到目标 → 「**撤销**」
3. 确认（不可恢复）

> 后端 API：
>
> - `POST /api/v1/system/admin/revoke`

列出所有

> - `GET /api/v1/system/admin/list`

---

4. 平台 API Key

> 与租户 API Key 独立，作用域是**整个平台**。

1. 进入 `/platform/system/platform-api-keys`
2. 「**+ 创建平台 API Key**」
3. 配置能力（可含 `system_*`）
4. **立即复制明文**（只显示一次）
5. 撤销：在列表 → 「**删除**」

> 后端 API：
>
> - `GET /api/v1/system/admin/api-keys`
> - `POST /api/v1/system/admin/api-keys`
> - `DELETE /api/v1/system/admin/api-keys/:key_id`

5. 运行时队列与 Worker Pool 治理

第 1 步：进入运行时队列

- 路径：`/platform/system/runtime-queues`
- 页面：`system/RuntimeQueues.vue`

第 2 步：查看池状态

池	说明
`core`	核心池（KB 解析、知识检索）
`post-process`	后处理池（嵌入、切片后处理）
`enrichment`	富化池（图谱、摘要）
`maintenance`	维护池（清理、回收）
Wiki 池（独立）	Wiki 摄取（`wiki_ingest*.go`）

第 3 步：per-model 并发治理

- 每个 LLM 模型有独立并发槽位
- 防止某个模型被高并发请求打满

第 4 步：查看任务

任务	操作
失败任务	重试 / 转 DLQ
DLQ 任务	查看 / 强制重跑 / 删除
pending 任务	检视（`task_inspector.go`）

> 详见 `docs/worker-pool-governance.md`

6. 平台审计日志

- 路径：`/platform/system/system-audit-log`
- 跨租户查询
- 与工作区审计日志一致，但**范围是全平台**
- 含去重（1 分钟滑动窗口）

---

7. 用户管理与审批

创建用户

> 后端 API：`POST /api/v1/system/admin/users/create`

重置密码

> 后端 API：`POST /api/v1/system/admin/users/reset-password`

跨租户审批

- Admin 限定（`RequireRoleOrSystemAdmin(Admin, …)`）：工作区内的审批
- System Admin：可看到所有租户的 pending 用户

---

8. 平台系统设置

- 路径：`/platform/system/system-global`
- 后端：`/api/v1/system/admin/settings[/:key]`
- 覆盖范围：集群级配置（默认存储配额、全局开关、Langfuse 集成等）

典型设置项：

key	说明
`default_storage_quota`	新租户默认存储配额
`enable_cross_tenant_access`	跨租户访问总开关
`langfuse_*`	Langfuse 集成（观测性）
`enable_oauth_*`	OIDC 集成
`enable_im_*`	IM 集成总开关

---

9. 关键不变式（System Admin 特别版）

1. **System Admin 切换租户时被授予临时 Admin**（`IsCrossTenantSuperuser` + `User.CanAccessAllTenants` 双重门控），**不能临时 Owner**。
2. **`RequireSystemAdmin` 不受 `EnableRBAC` 开关影响**——始终强制。
3. **跨租户访问需双重开启**：用户级 `can_access_all_tenants` **与** 集群级 `enable_cross_tenant_access` 都为 true。
4. **平台 API Key 与租户 API Key 是独立维度**——不可互相替代。
5. **孤儿租户自愈不影响 System Admin**——`resolveTenantRole` step3（孤儿自愈）只对 home tenant 的非平台用户生效。

---

10. 关键页面详解

10.1 平台设置（`/platform/system/system-global`）

Tab	说明
平台管理员	提升 / 撤销 / 列表
平台设置	集群级配置
存储配额	应用默认配额
平台审计	跳转到审计日志

10.2 运行时队列（`/platform/system/runtime-queues`）

区域	内容
顶部	各池概览（活跃 worker、积压任务）
中部	任务列表（pending / running / failed / DLQ）
底部	per-model 并发配置

10.3 待审批用户（`/platform/system/pending-users` 或工作区级）

> 跨租户聚合视图。

---

11. 常见问题（FAQ）

Q1：为什么我切到工作区后看不到「成员管理」按钮？

A：检查您是否被识别为 System Admin。`authStore.isSystemAdmin === true`。如果不行，刷新 JWT。

Q2：我能把其他租户的 Owner 降级吗？

A：不能。Owner-only 操作需要真实 Owner（您只是临时 Admin）。请联系工作区的真实 Owner。

Q3：平台 API Key 和租户 API Key 能混用吗？

A：不能。鉴权维度正交，跨作用域调用会被拒绝。

Q4：怎么把 System Admin 标志位从自己账号上移除？

A：需要**另一个**系统管理员撤销（自撤销有保护）。

Q5：运行时队列里任务一直 pending 怎么办？

A：检查 worker 是否启动、Redis 连接、模型并发槽位、DocReader 服务。详见 §5。

12. 应急操作清单（事故响应）

事故	操作
用户被锁（忘记密码 + 没 Admin）	`POST /system/admin/users/reset-password`
恶意成员被举报	工作区 Owner / Admin 移除；跨租户上报走平台审计
KB 被恶意删除	从 audit_log 查 → 异步 `ProcessKBDelete` 是异步任务，可能还在 queue，可尝试恢复备份（Postgres 备份）
API Key 泄露	立即 revoke → 创建新 Key → 通知所有调用方更新
DocReader 解析积压	重启 DocReader / 增加 worker / 检查存储
Worker pool 全挂	看 runtime-queues → 检查 Redis → 重启 app
System Admin 自己误操作	联系另一个 System Admin 撤销

13. 进一步阅读

- modules/06-system-settings.md § 5 平台配置
- `docs/worker-pool-governance.md` — Worker pool 治理
- `docs/Langfuse集成.md` — 观测性集成
- `docs/migration-troubleshooting.md` — 迁移故障排查
- README.md § 角色矩阵
- README.md § 关键不变式

模块参考 • 知识库（Knowledge Base）

> **对应后端服务**：`internal/application/service/knowledgebase*.go` + `knowledge_*.go` **对应前端**：`frontend/src/views/knowledge/*` **创建 / 编辑权限**：Contributor+（`g.Contributor()` 创，自己名下 `g.OwnedKBOrAdmin()` 改）

1. 知识库的三种类型

类型	用途	创建后主要操作
Document	上传 PDF / Word / Excel / 图片 / EPUB / XMind 等	上传文档 → 自动解析 → 切片 → 嵌入
FAQ	一问一答的标准问答对	手动添加问答；支持 Excel 批量导入
Wiki	自动摄取 + 手动维护的 Markdown 知识	Agent 自动摄取 + 手动编辑 + 修订历史

2. 创建知识库

路径

`/platform/knowledge-bases` → **「+ 新建知识库」**

表单字段

字段	说明	谁控制
名称	KB 显示名	Contributor
类型	Document / FAQ / Wiki	Contributor
描述	KB 范围	Contributor）
嵌入模型	哪种 Embedding 模型	默认走工作区配置（Admin 设置），可逐 KB 覆盖
向量库	pgvector / Qdrant / Milvus / Weaviate	默认走工作区配置
存储后端	MinIO / S3 / COS / OSS	默认走工作区配置
共享策略	私有 / 工作区内 / 跨工作区（Organization）	Owner 启用后可选

> **提交后** bootstrap 调用 `CreateKnowledgeBase`，异步清理任务 `ProcessKBDelete` 在删除时启动。

---

3. KB 详情页布局

```
/platform/knowledge-bases/:kbId

┌──────────────────────────────────────────────────────┐
│  KB 名称     ★收藏    ⚙ 设置   共享                  │
├──────────────┬───────────────────────────────────────┤
│  文件夹树     │  文档列表 / 卡片 / Wiki Tab  │
│              │                                       │
│  📁 产品      │  📄 产品白皮书 v2.docx      │
│  📁 客服      │  📄 FAQ.md                    │
│  📁 法规      │  📊 销售数据.xlsx           │
│              │                                       │
│  [+ 新建文件夹]│  + 上传                       │
└──────────────┴───────────────────────────────────────┘
```

4. 上传与解析

4.1 上传源

来源	说明
文件	拖拽或点击上传（PDF / Word / Excel / 图片 / EPUB / XMind / Markdown / HTML / MHTML / CSV / JSON）
URL	粘贴 URL，自动抓取
文件夹	批量上传整个文件夹
数据源	Feishu Wiki / Feishu Drive / Lark / GitLab / 腾讯 IMA / Notion / Yuque / RSS（详见 § 7）

4.2 解析流程

```
上传 → MinIO / S3 存储
     → asynq 任务派发 → DocReader gRPC（端口 50051）
     → DocReader 解析（PDF 渲染 → OCR / DOCX 结构提取 / Excel 表格 / …）
     → 返回结构化内容
     → 后处理：切片（按段落 / 固定窗口 / 语义）
     → 嵌入（Embedding 模型）
     → 写入向量库 + Postgres
     → 状态 done
```

4.3 支持的文档格式（DocReader）

格式	解析器
PDF（含扫描）	`pdf_parser.py`（扫描 PDF 渲染 JPEG → OCR）
DOCX	`docx_parser.py` / `docx2_parser.py` / `docx_merge.py`
Excel (XLSX)	`excel_parser.py` / `excel_convert.py` / `xlsx_merge.py`
PPTX	`ppt_convert.py` / `pptx_media.py`
图片	`image_parser.py`（OCR）
HTML / 网页	`html_parser.py` / `web_parser.py`
Markdown	`markdown_parser.py`
EPUB	`epub_parser.py`
MHTML	`mhtml_parser.py`
MarkItDown（Office 通用）	`markitdown_parser.py`
OpenDataLoader	`opendataloader_parser.py`
XMind	`xmind_parser.py`

4.4 特殊能力

- **扫描 PDF**：渲染为 JPEG（`DOCREADER_PDF_RENDER_MAX_WORKERS`、`DOCREADER_PDF_RENDER_DPI`、`DOCREADER_PDF_JPEG_QUALITY`）
- **MinerU**（高级解析）：可选 `MINERU_ENDPOINT`
- **多存储后端**：MinIO / S3 / 腾讯 COS / 阿里 OSS
- **gRPC + TLS + Token**：与 App 通信加密

4.5 排错

现象	可能原因	操作
上传后一直「pending」	asynq 队列阻塞 / Redis 异常	检查 Redis；`task_inspector` 看
一直「parsing」	DocReader 挂 / OOM / 配置错误	`docker logs WeKnora-docreader`
解析失败	文件超过 `MAX_FILE_SIZE_MB`（默认 50）	拆分 / 调高上限
中文乱码	解析器没装 OCR / Tesseract 中文包	重新构建镜像
嵌入失败	Embedding 模型不可达	测试模型连接

---

5. KB 设置的 13 个子面板

> KB 创建者（`g.OwnedKBOrAdmin()`）可以修改；Admin+ 可修改工作区默认值。

面板	用途
KBAdvancedSettings	高级设置（混合检索策略、MMR、阈值）
KBChunkingSettings	切片策略（按段落 / 固定窗口 / 语义）
KBChunkingDebug	切片调试（看每段切分结果）
KBIndexingStrategy	索引进阶（标签过滤、graph 增强）
KBModelConfig	嵌入模型覆盖
KBParserSettings	解析器选择（默认 / MinerU / MarkItDown / OpenDataLoader）
KBStorageSettings	存储后端覆盖
KBVectorStoreSettings	向量库覆盖
KBShareSettings	共享策略
GraphSettings	知识图谱启用
DataSourceSettings	数据源配置
DataSourceSyncLogs	数据源同步日志
KnowledgeBaseActivitySettings	KB 活动审计

> 完整代码：`frontend/src/views/knowledge/settings/*`

---

6. 检索原理（`HybridSearch`）

> 来源：`knowledgebase_search.go` + `knowledgebase_search_fanout.go` + `knowledgebase_search_fusion.go` + `knowledgebase_search_faq.go`

```
查询 → 嵌入查询 → 多个向量库扇出
                      ↓
                + BM25 关键词搜索
                + FAQ 精确匹配
                + 多 KB 跨库检索
                      ↓
                Rerank 重排
                      ↓
              阈值过滤 + MMR 去冗余
                      ↓
                Fusion 多源融合
                      ↓
              返回 Top-K chunks
```

关键参数（KB 高级设置）：

参数	作用
`top_k`	返回几个 chunk
`score_threshold`	最低相关分
`mmr_lambda`	MMR 多样性（0-1）
`rerank_enabled`	是否 rerank
`hybrid_weight`	向量 / 关键词权重（0-1）

---

7. 数据源

> 来源：`internal/application/service/datasource_service.go` + `internal/datasource/ConnectorRegistry`

支持的数据源

数据源	用途
Feishu Wiki	飞书知识库
Feishu Drive	飞书云盘
Lark Wiki / Drive	Lark（国际版）
GitLab	仓库 Wiki / Markdown
腾讯 IMA	腾讯智能助手知识库
Notion	Notion 数据库 / 页面
Yuque	语雀
RSS	任意 RSS 源

配置步骤

1. KB 详情 → 设置 → 数据源
2. 「**+ 添加数据源**」
3. 选 connector → 填凭证 → 测试
4. 启用同步（周期 / 手动）
5. 在「同步日志」看进度与失败

> 详见 `docs/数据源导入开发文档.md`

---

8. FAQ 维护

> FAQ 类型 KB / 固定 KB 中的 FAQ Tab。

- **单个添加**：问答对（问、答、tags）
- **批量导入**：上传 Excel / CSV（标准列：question / answer / tags）
- **批量导出**：同步 `*_faq_export.xlsx`
- **批量打标签**：选多条 → 批量打

9. 知识图谱（Graph）

> 见 `docs/开启知识图谱功能.md` + `docs/KnowledgeGraph.md`

- 启用 GraphSettings 后，KB 会抽取实体与关系
- 在 KB 详情中可看交互式知识图谱
- 与 Wiki 模式配合，自动构建结构化知识

---

10. 标签与文件夹

标签

- 范围：KB 内 / 跨 KB
- 自动打标：`knowledge_auto_tag.go`（解析完成后从现有标签集挑匹配，不创建新标签，不覆盖手工标签）
- 批量打：`BatchTagDialog.vue`

文件夹

- 多层文件夹树（`KbFolderTree.vue`）
- 拖拽移动文档
- 文件夹权限继承（KB 创建者可管理）

---

11. 共享

工作区内共享

- 默认所有工作区成员可访问 KB（按 Viewer 默认放行读）
- 「**仅自己**」可选：只有创建者可看

跨工作区共享

- 由 Owner 启用
- 通过 Organization 共享

- 按 `OrgMemberRole` 裁剪（admin / editor / viewer）
- 详见 roles/04-owner.md § 7

---

12. 复制 / 收藏 / 删除

操作	谁可以做	行为
复制	Viewer（任何人）	生成副本，您是创建者，默认私有
收藏	Viewer（任何人）	加入左侧「我的」
删除	KB 创建者 / Admin+	软删除 `deleted_at`；异步 `ProcessKBDelete` 清理

---

13. 关键不变式

1. **KB 删除是异步的**——列表立即消失，资源清理在 asynq 后台任务。
2. **复制 KB 自动改名**「原名 的副本」。
3. **共享 KB 的写入工具被过滤**——共享给其他租户时，写操作工具自动隐藏（只读语义）。
4. **自动标签不创建新标签**——只从现有标签集挑匹配。

---

14. 进一步阅读

- `docs/CHUNKING.md` — 切片策略
- `docs/使用其他向量数据库.md` — 接入其他向量库
- `docs/KnowledgeGraph.md` — 知识图谱
- `docs/数据源导入开发文档.md` — 数据源开发
- `docs/migration-troubleshooting.md` — 迁移排错
- roles/02-contributor.md — Contributor 视角
- roles/03-admin.md § 3.2 — Admin 视角下 KB 配置
- modules/02-agent.md — Agent 如何使用 KB

模块参考 • 智能体（Agent）

> **对应后端**：`internal/application/service/agent_service.go` + `custom_agent.go` + `internal/agent/tools/*` **对应前端**：`frontend/src/views/agent/*` **创建 / 编辑权限**：Contributor+ 创，自己名下 + Admin+ 改

---

1. 智能体的概念

LuoSA 的「智能体」= **绑定 KB + 工具 + 系统提示 + 模型的 AI 实体**，可以做单轮推理（ReAct 循环）。

```
用户提问
  ↓
Agent（system prompt + model）
  ↓
LLM 思考 → 调用工具 → 看结果 → 再思考 → ...
  ↓
最终回答（带引用）
```

---

2. 内置智能体（`config/builtin_agents.yaml`）

> 启动时由 `cmd/server/bootstrap.go` 自动 upsert 到 DB。所有工作区都有。

ID	中文名	display_group	用途
`builtin-quick-answer`	默认对话	chat	直接 LLM 聊天，不依赖 KB
`builtin-doc-review-proofread-chat`	文档与招投标审查	chat	四维度审查 + 招投标对比，自动生成修订 docx
`builtin-doc-generation-chat`	投标文件编写	chat	基于招标文件评分项生成应答，可转 docx

每个内置 agent 支持 default / zh-CN / zh-TW / ja-JP / ko-KR 多语言 i18n。

3. Agent 类型与预设（`config/agent_type_presets.yaml`）

`smart-reasoning` 模式下选择 `agent_type` 时，预设会自动填入：

- 推荐系统提示
- 工具 allowlist
- KB 过滤规则（any_of / all_of / none_of，按 capabilities：vector / keyword / wiki / graph / faq）

agent_type	典型场景
`customer-service`	客服问答
`document-qa`	文档检索 + 回答
`data-analysis`	数据分析（需要 data_analysis 工具）
`code-review`	代码审查
`research`	研究助手

4. Agent 编辑器（左导航分组）

`/platform/agents` → 点击 Agent → 编辑器打开，**左侧导航**包含：

分组	内容
基础	名称、描述、type、display_group、icon、tags
提示词	system prompt + welcome message + 推荐问题
模型	temperature / max_tokens / 模型覆盖
知识库	选择 KB + 选择具体文档 + capability 过滤
工具	知识检索 / 文件操作 / 数据 / MCP / Skill / 多模态
技能	启用 Skill 列表（`SkillsEnabled` + `SkillsSelectionMode`）
沙箱	选择沙箱后端（Docker / E2B / Cube）+ 网络策略
对话策略	temperature / max_tokens / follow-up
问题建议	推荐问题管理

> 前端代码：`frontend/src/views/agent/AgentEditorModal.vue`

---

5. Agent 能调用的工具（`internal/agent/tools/`）

5.1 知识检索

工具	作用
`knowledge_search`	KB 混合检索（向量 + 关键词 + FAQ）
`grep_chunks`	按关键词搜索 chunk
`list_knowledge_chunks`	列出文档的所有 chunk
`get_document_info`	查文档元信息
`faq_snippet`	FAQ 精确匹配

5.2 文件操作

工具	作用
`kb_get_file`	拿 KB 内原始文件（PDF / DOCX / XLSX）
`kb_get_image`	拿 KB 内的 figure / 表格

5.3 Wiki

工具	作用
`wiki_*`	Wiki 模式专用工具（读取页面、列表、图谱）

5.4 数据

工具	作用
`data_analysis`	DuckDB SQL 查询 + 数据分析
`database_query`	数据库查询
`data_schema`	查数据 schema

5.5 多模态

工具	作用
`imagefetch/*`	图片处理
`image_markdown_output`	图片转 Markdown

5.6 其他

工具	作用
`json_repair`	JSON 修复
`normalize_id`	ID 规范化
`output_budget`	输出预算控制
`mcp_tool`	MCP 工具调用
`mcp_oauth`	MCP OAuth 中途授权
`file_mutation_queue`	文件变更队列

---

6. Agent 类型与 Skill / Sandbox 集成

SkillsEnabled

- 启用哪些 Skill 在该 Agent 中可用
- 选择模式：
  - `all`：所有启用的 Skill
  - `whitelist`：仅白名单
  - `mention`：通过 `@Skill` 提及启用（按会话范围）

SkillsSelectionMode

- `mention` 模式：`@doc-coauthoring` 在对话中被识别并启用

Sandbox

- 后端：Docker（默认）/ E2B / Cube

- 网络策略：默认限制出网
- TTL：会话级沙箱生命周期

---

7. Agent 服务核心流程（`CreateAgentEngine`）

> 来源：`internal/application/service/agent_service.go` `CreateAgentEngine`

```
1. 注册 MCP 工具（pinned MCP service）
2. 解析 KB 与文档上下文（kbRetrievalCapabilities）
3. 注册沙箱文件工具（如果 sandbox 启用）
4. 注册 Skill 文件工具（read_skill / write_skill_file）
5. 注册沙箱 shell 工具（如果允许）
6. 解析 pinned Skill（loadInstalledSkillBundle）
7. userEnvResolver + skillEnvCapture（个人 / 工作区 env）
8. 注册最终工具集
9. ValidateConfig
10. 构建 ReAct 引擎
```

---

8. 共享 Agent

> 跨工作区共享 Agent，按 `OrgMemberRole` 裁剪。

- **Owner** 启用共享
- **共享 Agent 的写工具被过滤**（`filterSharedAgentWriteTools`）：跨租户只读语义
- 在 Agent 详情 → 「**共享**」Tab 配置

9. Agent 的运行生命周期

```
用户发起对话
  ↓
会话层（session_agent_qa.go）
  ↓
CreateAgentEngine（懒构建）
  ↓
ReAct 循环
  ├─ LLM 思考 → 输出 tool_call
  ├─ 工具执行 → 返回结果
  ├─ 再次 LLM 思考 → ...
  └─ 最终回答
  ↓
流式输出到前端（chunked transfer / WebSocket）
  ↓
持久化到 session_messages
```

---

10. 调试与测试

10.1 Agent 编辑器内测试

- 点击「**测试**」按钮 → 右侧对话面板
- 输入问题 → 看流式输出
- 看调用的工具与参数（在面板中显示）

10.2 单步调试

- 部分工具支持「**单步运行**」（测试工具本身）
- 例如 `data_analysis` 工具：直接输入 SQL 测试

10.3 Langfuse 集成（观测性）

- 后端配置 `LANGFUSE_*` 环境变量
- Agent 运行 trace 上传到 Langfuse
- 看 LLM 延迟、token 用量、工具调用

> 详见 `docs/Langfuse集成.md`

11. 自定义 Agent 的最佳实践

1. **从 agent_type_presets 选类型**：自动填入推荐配置
2. **系统提示通用化**：不绑特定租户 / KB / 文档（遵循 agent-prompt-contract）
3. **KB 选择模式**：
   - 固定 1-2 个 KB：精确场景
   - 任意 KB（`any_of`）：开放场景
   - 按 capability 过滤：保证 KB 有 vector / faq / graph
4. **工具最小化**：只启用必需工具（减少 token 与误用）
5. **测试后再发布**：通过「测试」按钮多轮对话验证
6. **配置 follow-up**：自动推荐下一步问题
7. **共享时注意只读**：跨工作区共享会被过滤写工具

---

12. 关键不变式

1. **共享 Agent 的写入工具被过滤**——确保只读语义。
2. **`SkillsSelectionMode=mention` 时**，必须在对话中 `@Skill_name` 才会启用。
3. **沙箱 shell_exec 仅在 sandbox 启用且 `SkillsEnabled` 配置允许时可用**。
4. **Agent Type 预设不可关闭**——选择 type 时会自动应用。
5. **内置 Agent 不能删除**——只能「**重置为内置**」。

---

13. 进一步阅读

- modules/05-skills-mcp.md — Skills / MCP 详细
- modules/03-workflow.md — 工作流（多节点编排）
- modules/04-session-chat.md — 会话与聊天
- roles/02-contributor.md — 创建第一个 Agent
- `config/builtin_agents.yaml` — 内置 Agent 配置
- `config/agent_type_presets.yaml` — Agent 类型预设
- `docs/contributing/agent-prompt-contract.md` — Prompt 合约

模块参考 • 工作流（Workflow）

> **对应后端**：`internal/application/service/workflow_service.go` + `internal/agent/workflow/*` **对应前端**：`frontend/src/views/workflow/*` **创建 / 编辑权限**：Contributor+（`g.Contributor()` 创，自己名下 + Admin+ 改） **发布**：需要 DSL 可编译

---

1. 工作流是什么

LuoSA 工作流是 **Dify-parity** 的可视化节点编排（DAG）。把多个 LLM 调用、检索、代码、Agent、HTTP、MCP、Switch 等节点拼成一个流程。

```
[Start] → [Retrieval] → [LLM] → [Switch] → [Answer]
                                    ↓
                              [Code] → [HTTP] → [MCPTool]
```

适用场景：

- 多步骤业务流程（工单分类 → 查 KB → 调 HTTP → 写数据库）
- 复杂的 RAG（多次检索 + 重排 + 引用）
- 跨系统集成（多个 MCP 服务）
- 可视化的 Agent 编排

---

2. 编辑器（`/platform/workflow/:id/edit`）

> Dify 风格画布编辑器：`frontend/src/views/workflow/WorkflowEditor.vue`

2.1 工具栏

按钮	功能
变量抽屉	编辑工作流级变量
自动布局	自动整理节点位置
复制/粘贴	节点复制粘贴
撤销/重做	编辑历史
导入/导出 DSL	JSON 格式
自动保存	时间戳显示
保存	显式保存
发布/重新发布	冻结 DSL 为 published 版本
运行抽屉	打开运行历史

2.2 节点组件（`workflow/components/`）

节点	来源	用途
Start	builtin	入口节点
Answer	builtin	终端输出
LLM	builtin	LLM 调用
Retrieval	builtin	KB 检索
Switch	builtin	条件分支
QuestionClassifier	advanced	问题分类
ParameterExtractor	advanced	参数抽取
WebSearch	advanced	网络搜索
Iteration	advanced	迭代
Template	extended	模板渲染
VariableAggregator	extended	变量聚合
HTTP	extended	HTTP 请求
DataOps	extended	数据操作（SQL / DuckDB）
Code	code	代码节点（Python / JS / SQL）
Agent	agent	引用平台智能体
MCPTool	mcp_tool	MCP 工具调用

3. DSL（领域特定语言）

3.1 结构（v1）

```
{
  "version": 1,
  "components": [
    { "id": "start", "component_name": "Start", "data": {...} },
    { "id": "retrieval", "component_name": "Retrieval", "data": {...} },
    { "id": "llm", "component_name": "LLM", "data": {...} },
    { "id": "answer", "component_name": "Answer", "data": {...} }
  ],
  "graph": {
    "edges": [
      { "source": "start", "target": "retrieval" },
      { "source": "retrieval", "target": "llm" },
      { "source": "llm", "target": "answer" }
    ]
  },
  "variables": [
    { "name": "user_query", "type": "string", "required": true }
  ]
}
```

3.2 校验规则

- `version == 1`
- `components` 非空
- 每个组件有 `component_name`
- 至少一个 Start 节点
- 边的 source / target 节点存在

> 校验函数：`ValidateWorkflowDSL`

3.3 导入 / 导出

- **导出**：工具栏 → 「**导出 DSL**」→ JSON 文件
- **导入**：工具栏 → 「**导入 DSL**」→ 选择 JSON 文件 → 合并到画布

4. 发布模型

状态	含义
draft	草稿，可以继续编辑
published	已发布，运行时会用这个版本的 DSL（冻结）
archived	归档，不可编辑、不可运行

4.1 发布流程

1. 编辑工作流
2. 点击「**保存**」（保存当前 draft）
3. 点击「**发布**」（`PublishWorkflow`）：
    - 校验 DSL 是否可编译
    - 冻结 DSL 快照为 published 副本
    - `dslForRun` 切换到 published 版本

4.2 状态切换

- `SetWorkflowStatus`：draft ↔ archived（**不能从这里直接 published**）
- 草稿 / 归档运行：仅创建者或 Admin 可调试运行

---

5. 运行与调度

5.1 运行

入口	说明
编辑器「**运行**」按钮	调试运行
「**运行抽屉**」	看运行历史
API `POST /workflows/:id/run`	程序化触发

> 后端：`RunWorkflow`（同步 / 异步可切换）；asynq 任务 `workflow:run`

5.2 单节点调试

> `RunWorkflowNode`：单独跑某个节点，输入手动指定。

5.3 调度（Cron）

- `workflow_scheduler.go` + `workflow_schedule_service.go`
- Cron 表达式（如 `0 9 * * 1-5` 工作日 9 点）
- 触发时调用 `RunWorkflow`

5.4 Checkpoint（断点）

- `workflow_checkpoint.go`：保存运行中间状态
- `ResumeWorkflowRun`：从 checkpoint 恢复
- `CancelWorkflowRun`：取消运行

5.5 事件流（Redis）

- `workflow_run_events.go` + Redis 通道 `workflow:run:<id>`
- `SubscribeWorkflowRunEvents`：订阅节点级事件
- 前端通过 SSE / WebSocket 看节点进度

---

6. 节点适配器

> `internal/agent/workflow/nodes_adapters.go`

函数	作用
`runLLM` / `runLLMStream`	LLM 节点
`runRetrieval` / `rerankChunks`	检索 + 重排
`runAgent` / `runPlatformAgent`	Agent 节点
`runMCPTool`	MCP 节点
`runCode`	代码节点
`runWebSearch`	Web 搜索
`attachmentPrompt`	附件 prompt

7. 节点配置（典型）

7.1 LLM Node

字段	说明
`model`	模型选择
`system_prompt`	系统提示
`user_prompt`	用户提示（支持变量）
`temperature`	0-1
`max_tokens`	最大 token
`stream`	是否流式

7.2 Retrieval Node

字段	说明
`kb_ids`	KB ID 列表
`top_k`	检索条数
`score_threshold`	最低分
`rerank`	是否重排
`hybrid_weight`	向量 / 关键词权重

7.3 Switch Node

- 多分支条件（变量比较 / 同基真值）
- 默认分支（兜底）

7.4 Code Node

- 多语言：Python / JavaScript / SQL
- 输入：上节点输出（变量）
- 输出：返回的 dict

7.5 HTTP Node

- URL / Method / Headers / Body
- 鉴权：None / API Key / Bearer / Basic
- 响应处理：JSON / Raw / Stream

---

8. 关键 API

Method	路径	用途
POST	`/workflows`	创建
GET	`/workflows/:id`	详情
GET	`/workflows`	列表
PUT	`/workflows/:id`	更新
DELETE	`/workflows/:id`	删除
POST	`/workflows/:id/publish`	发布
PUT	`/workflows/:id/status`	切换 draft/archived
POST	`/workflows/:id/run`	运行
POST	`/workflows/:id/nodes/:node_id/run`	单节点运行
POST	`/workflows/:id/runs/:run_id/cancel`	取消运行
POST	`/workflows/:id/runs/:run_id/resume`	恢复运行
GET	`/workflows/:id/runs`	运行列表
GET	`/workflows/:id/runs/:run_id/events`	订阅事件（SSE / WS）

---

9. 最佳实践

1. **先画图，再写 DSL**：用画布拖拽，再导出 DSL 复用
2. **变量命名规范**：`snake_case`、明确含义
3. **避免过深嵌套**：Switch / Iteration 嵌套不要太深
4. **复用**：常用子图抽出来，用 Agent 节点调用

5. **测试单节点**：用「单节点调试」先验证逻辑
6. **发布前校验**：发布按钮会触发编译检查
7. **看事件流**：运行时用 SSE 订阅，看节点执行进度

---

10. 关键不变式

1. **状态 draft/archived 不能直接从 API 切到 published**——必须走 `PublishWorkflow`。
2. **运行使用 published DSL**——draft 修改不影响已运行实例。
3. **checkpoint 在 cancel/resume 间保持**。
4. **节点 ID 在工作流内唯一**——拖拽复制会自动加后缀。

---

11. 进一步阅读

- modules/02-agent.md — Agent 节点使用
- modules/05-skills-mcp.md — MCP 节点使用
- `config/workflows/` — 内置工作流示例
- `internal/agent/workflow/` — 工作流引擎源码
- roles/02-contributor.md — 创建第一个工作流

模块参考 • 会话与聊天（Session / Chat）

> **对应后端**：`internal/application/service/session*.go` **对应前端**：`frontend/src/views/chat/*` + `creatChat/*` + `embed/*` **使用权限**：所有角色（Viewer / Contributor / Admin / Owner / System Admin） **创建会话**：需要 Contributor+（`g.Contributor()`）

1. 概念

- **会话（Session）**：一组用户消息 + Agent 回复的对话线程
- **来源（source）**：Web / API / Embed / IM（10 个平台）
- **状态**：active / archived
- **主体（principal）**：用户 / IM 用户 / API Key / Embed 访客

2. 三类入口

入口	路径	说明
新建对话首页	`/platform/creatChat`	推荐问题卡片 + 输入框
对话主界面	`/platform/chat/:chatid`	完整对话视图
Embed widget	`/embed.html`	网站嵌入（独立 HTML）

3. 新建对话首页（`/platform/creatChat`）

区域	内容
顶部	Agent 选择器（默认 `builtin-quick-answer`）
主体	推荐问题卡片（按 `source === 'faq'` 标记）
底部	输入框
侧边	KB 编辑器入口（Contributor+）

典型交互：

```
1. 选 Agent → 2. 提问 → 3. 看回答 → 4. 追问 → 5. 收藏会话
```

---

4. 对话主界面（`/platform/chat/:chatid`）

4.1 区域

```
┌──────────────────────────────────────────────────────┐
│  Agent 选择器  | 会话标题 | 参考资料 | ⋮ 菜单        │
├──────────────────────────────────────────────────────┤
│                                                      │
│  👤 用户消息（Markdown）                             │
│                                                      │
│  🤖 助手消息（Markdown）                             │
│      [1] [2] [3] ← 引用 popover                      │
│      —— 来源：销售手册 v2.docx                       │
│                                                      │
│  --- 追问 ---                                        │
│                                                      │
│  💡 Follow-up 建议：                                 │
│      → 相关问题 1                                    │
│      → 相关问题 2                                    │
│                                                      │
├──────────────────────────────────────────────────────┤
│  [📎附件] 输入框                                     │
└──────────────────────────────────────────────────────┘
```

4.2 关键元素

元素	说明
引用 popover	回答下方的 `[1] [2]`，hover 看原文片段，点击跳到 chunk 详情
参考资料抽屉	完整引用列表（KB / 文档名 / chunk ID / 得分）
Follow-up	自动推荐下一步问题
附件	上传临时文件（图片 / 文档），自动 Agent 读取
IM 来源标识	显示该会话来自哪个 IM 平台
时间戳	大纲

---

5. 会话后台管线

5.1 知识问答管线（`session_knowledge_qa.go`）

```
用户提问 → RAG 检索（HybridSearch）
         → 拼装 Prompt（带检索结果）
         → LLM 生成
         → 流式输出到前端
         → 持久化到 session_messages
```

5.2 智能体问答管线（`session_agent_qa.go`）

```
用户提问 → CreateAgentEngine
         → ReAct 循环
           ├─ LLM 思考 → tool_call
           ├─ 工具执行 → 结果
           ├─ 再次思考 → ...
           └─ 最终回答
         → 流式输出
```

5.3 临时附件（`session_attachment_staging.go`）

- 会话作用域 image/doc 暂存
- 异步解析（DocReader）
- 解析后注入 Agent / KB 上下文

5.4 沙箱绑定（`session_sandbox_pin.go`）

- 会话级沙箱生命周期管理
- 同一会话内复用同一沙箱
- 会话结束 / 超时销毁

---

6. 会话主体治理（关键不变式）

> 来源：`loadSessionForRead` + `runtimeMayBypassAdminConsoleRead`

主体	能读哪些会话
普通用户	只能读自己拥有的会话（`owner_id == user.id`）
Admin 控制台	可读所有会话（管理用）
IM 用户	只能读自己 IM 上下文产生的会话
API Key 用户	只能读自己创建的（按 KB allow-list）
Embed 访客	只能读 Embed 通道会话

> **这是安全边界**——IM 通道会话不能跨主体读。

---

7. 会话操作

操作	谁能做	API
创建会话	Contributor+	`CreateSession`
列出会话	自己拥有的	`GetSessionsByTenant` / `ListSessions`
收藏 / Pin	自己拥有的	`SetSessionPinned`
改标题	自己拥有的	`GenerateTitle` / 显式改
删除会话	自己拥有的 / Admin+	`DeleteSession`
批量删除	自己拥有的	`BatchDeleteSessions`
删除全部	自己拥有的	`DeleteAllSessions`

8. 跨主体通道

8.1 Embed 网站 widget（`/embed.html`）

> 前端：`frontend/src/views/embed/*`，后端：`embed_channel.go` / `embed_webhook.go` / `embed_session.go`

特点：

- **安全模式**（推荐）：token 交换 + 速率限制
- **Publish token**：前缀 `em_`，32 字节随机
- 访客通过 widget 与 Agent 对话，会话归属于 publish token 所属 KB

配置（Admin / Owner）：

1. 设置 → 集成 → Embed → 配置 publish token、subdomain、域名白名单
2. 在前端页面嵌入：

```
<script src="https://your-domain/embed.js" data-token="em_xxx"></script>
```

> 详见 `docs/embed-secure-mode.md` 与 `docs/embed-subdomain.md`

8.2 IM 集成（10 个平台）

> 详见 `docs/IM集成开发文档.md`

支持：WeCom（企微）/ Feishu（飞书）/ Lark（国际版）/ QQBot / Slack / Telegram / DingTalk（钉钉）/ Mattermost / WeChat（微信）/ Yunzhijia（云之家）

配置：Admin → 设置 → Integrations → 选择平台 → 填凭证 → 启用

8.3 REST API

> 程序化调用，API Key 鉴权。详见 modules/06-system-settings.md § 3 租户 API Key

```
curl -X POST https://your-domain/api/v1/chat \
  -H "X-API-Key: tk_xxx" \
  -H "Content-Type: application/json" \
  -d '{"agent_id": "...", "kb_ids": [...], "message": "..."}'
```

9. 推荐问题（来源）

推荐问题分两层：

层	来源
KB 推荐问题	由 KB 内 FAQ 条目生成（`source === 'faq'`）
对话推荐	上轮回答末尾的 follow-up 建议（由 LLM 生成）

10. 长期记忆（Memory）

- 跨会话记住用户偏好
- 类别：profile / preference / fact / task / interest
- 自动抽取 + 用户确认（`search_memory` 工具）
- 设置：`/platform/settings?section=memory`

11. 聊天历史设置

- 路径：`/platform/settings?section=chathistory`
- 配置保留策略（天数 / 永久）
- 软删除 vs 硬删除策略
- 跨租户隔离

12. 关键不变式

1. **会话是按主体隔离的**——IM 用户不能读到 Admin 控制台创建的会话。
2. **Embed 通道会话安全模式推荐**——避免 token 泄露。
3. **跨租户超管能读到所有会话**（admin bypass）。
4. **会话标题可异步生成**（`GenerateTitleAsync` + `sanitizeGeneratedTitle`）。
5. **沙箱与会话绑定**——会话结束销毁沙箱。

13. 进一步阅读

- `docs/IM集成开发文档.md` — IM 集成开发文档
- `docs/embed-secure-mode.md` — Embed 安全模式
- `docs/embed-subdomain.md` — Embed subdomain
- modules/01-knowledge-base.md § 6 — HybridSearch 原理
- modules/02-agent.md § 9 — Agent 运行生命周期
- modules/06-system-settings.md § 6-7 — API Key / OIDC / IM

模块参考 • 技能与 MCP（Skills / MCP）

> **对应后端**：
>
> - Skills：`internal/application/service/skill_service.go` + `tenant_skill_*.go`
> - MCP：`mcp_service.go` + `mcp_oauth.go` + `mcp_tool_approval_service.go`
> - 沙箱：`tenant_sandbox_config.go` + `sandbox_*` **对应前端**：`/platform/settings?section=skills` + `?section=mcp` + `?section=sandbox` **管理权限**：Admin+（`g.Admin()`）用于安装工作区级；Contributor 在 Agent 内启用

---

1. Skills（技能）

1.1 概念

Skill = 一个可被 Agent 调用的能力单元（一个目录 + `SKILL.md` + 可选脚本）。

```
skills/preloaded/
├── citation-generator/
│   ├── SKILL.md           ← YAML frontmatter + Markdown 说明
│   └── scripts/            ← 可选
│       └── cite.py
├── data-processor/
│   ├── SKILL.md
│   └── scripts/
│       └── process.py
├── doc-coauthoring/
│   └── SKILL.md
├── document-analyzer/
│   └── SKILL.md
├── office-template-fill/
│   ├── SKILL.md
│   └── scripts/
│       └── fill.py
└── openmaic-classroom/
    └── SKILL.md
```

1.2 SKILL.md 格式

```
---
name: my-skill              # kebab-case, ≤ 64 字符
description: 一句话描述
metadata:
  type: builtin | tenant
---

# Skill 说明

详细描述 Agent 何时使用、如何使用。

## 工具列表
- `tool_name`：用途

## 脚本
```bash
python scripts/run.py <args>
```
```

1.3 命名规则

- **kebab-case**
- 长度 ≤ 64 字符
- 仅含 `[A-Za-z0-9_-]`

1.4 安装源

> `tenant_skill_install.go` + `tenant_skill_source.go`

源	说明
ClawHub	官方市场
SkillHub	社区市场
Git	直接 Git URL
Zip	上传 zip 包

1.5 安装范围

- **工作区级**（Admin 操作）：所有成员可用
- **个人级**（自己）：仅自己可用

1.6 安装流程

1. `/platform/settings?section=skills`
2. 「**+ 安装技能**」→ 选择源
3. 配置：
   - **来源 URL** / **上传文件**
   - **安装路径**：工作区 / 个人
   - **环境变量**（可选）
4. 安装过程显示进度 → 完成
6. 在 Agent 编辑器 → Skill 标签中启用

1.7 在 Agent 中使用

- **Agent 编辑器** → 「**技能**」标签
- 配置 `SkillsEnabled`：启用哪些 Skill
- 配置 `SkillsSelectionMode`：
  - `all`：全部启用
  - `whitelist`：白名单
  - `mention`：在对话中 `@Skill_name` 启用

1.8 脚本执行（沙箱）

- Skill 脚本在 **sandbox 容器**中执行
- Docker 镜像预装：`python-docx` / `openpyxl` / `pandas` / …（见 `docker/Dockerfile.sandbox`）
- 沙箱后端：Docker（默认） / E2B / Cube
- 网络策略：默认限制出网

```
- TTL：会话级生命周期

### 1.9 文件工具

Agent 可调用：

| 工具 | 作用 |
|---|---|
| `read_skill` | 读 SKILL.md |
| `write_skill_file` | 写 skill 文件 |
| `execute_skill_script` | 执行 skill 脚本（沙箱内） |
| `shell_exec` | shell 执行（沙箱内） |

### 1.10 维护操作

| 操作 | 说明 |
|---|---|
| 更新 | 重新安装（覆盖） |
| 卸载 | 解除 Skill 安装（`tenant_skill_remove.go`） |
| 回收运行 | `tenant_skill_reaper.go`（清理超时 / 异常进程） |
| 停止运行 | `tenant_skill_stop.go` |

### 1.11 内置 Skills（`skills/preloaded/`）

| Skill | 用途 |
|---|---|
| `citation-generator` | 生成引用 |
| `data-processor` | 数据处理 |
| `doc-coauthoring` | 文档协作 |
| `document-analyzer` | 文档分析 |
| `office-template-fill` | Office 模板填充 |
| `openmaic-classroom` | 课堂场景 |

---

## 2. MCP（Model Context Protocol）

### 2.1 概念

**MCP** 是 Anthropic 提出的开放协议，让 Agent 调用外部工具。MCP 服务器暴露一组「工具」，Agent 通过统一协议调用。

### 2.2 架构
```

LuoSA App（Agent） ↓ MCP 协议（stdio / SSE / HTTP） MCP Server（独立服务 / 第三方） ↓ 外部系统（数据库 / SaaS API / …）

```
### 2.3 内置 MCP Server（`mcp-server/`）

> 独立 Node.js 服务，40 KB+ `weknora_mcp_server.py`，29 个工具，支持 stdio / SSE / HTTP。

- 发布到 PyPI：`tencent-weknora-mcp`
- v2.x 高级 API 迁移，新增 `create_knowledge_from_text` 和 `list_shared_knowledge_bases`

### 2.4 配置 MCP 服务（Admin）

1. `/platform/settings?section=mcp`
2. 「**+ 新建 MCP 服务**」
3. 配置：

| 字段 | 说明 |
|---|---|
| **名称** | 显示名 |
| **传输协议** | stdio / SSE / HTTP |
| **端点** | URL 或命令 |
| **认证** | None / API Key / OAuth2 |
| **凭证** | API Key / OAuth secret |

4. 点击「**测试连接**」→ 拉取工具列表
5. 保存 → 在 Agent 编辑器中可用

### 2.5 OAuth2 中途授权（`mcp_oauth.go`）

- Agent 在对话中调用需要授权的工具
- 自动触发 OAuth flow（mid-conversation）
- 用户看到授权弹窗 → 同意 → 继续调用

### 2.6 工具审批（`mcp_tool_approval_service.go`）

- **模式**：手动允许 AI 干预
- Agent 调用 MCP 工具前弹出审批框
- 用户批准 / 拒绝

### 2.7 在 Agent 中使用 MCP

- Agent 编辑器 → 「**工具**」标签
- 启用 MCP 服务（pinned）
- `resolvePinnedMCPServiceInfos` + `attachPinnedMCPToolNames`
- Agent 自动获得 MCP 工具列表

### 2.8 关键 API

| Method | 路径 | 用途 |
|---|---|---|
| POST | `/mcp/services` | 创建 MCP 服务 |
| GET | `/mcp/services` | 列表 |
```

GET	`/mcp/services/:id`	详情
PUT	`/mcp/services/:id`	更新
DELETE	`/mcp/services/:id`	删除
POST	`/mcp/services/:id/test`	测试连接
GET	`/mcp/services/:id/tools`	工具列表
PUT	`/mcp/services/:id/credentials`	更新凭证
DELETE	`/mcp/services/:id/credentials`	清除凭证
GET	`/mcp/services/:id/resources`	资源列表

---

3. 沙箱（Sandbox）

3.1 概念

Skill 脚本 + Shell 执行的隔离环境。

3.2 后端

后端	适用
Docker（默认 / 集群）	本地 / 自建集群
E2B	云端托管
Cube	第三方托管

3.3 配置（Admin）

1. `/platform/settings?section=sandbox`
2. 选后端 → 填凭证 / socket / endpoint
3. 配置：

字段	说明
`network_policy`	网络策略（默认限制出网）
`ttl`	会话沙箱 TTL
`cpu_limit` / `memory_limit`	资源限制

4. 保存

3.4 集群模式（`sandbox-cluster.md`）

- 多节点 Docker 集群
- 负载均衡
- 故障转移

---

4. 关键不变式

1. **Skill 名称 kebab-case 且唯一**。

```
2. **工作区 Skill 由 Admin 安装，个人 Skill 自己安装**。
3. **沙箱默认限制出网**（`network_policy`）。
4. **MCP OAuth 中途授权需要用户确认**。
5. **共享 Agent 的写工具被过滤**——包括 Skill 文件写、Skill 脚本执行（视配置）。

---

## 5. 进一步阅读

- [`docs/BUILTIN_MCP_SERVICES.md`](../../BUILTIN_MCP_SERVICES.md) — 内置 MCP 服务
- [`docs/MCP功能使用说明.md`](../../MCP功能使用说明.md) — MCP 功能说明
- [`docs/sandbox-protocol.md`](../../sandbox-protocol.md) — 沙箱协议
- [`docs/sandbox-cluster.md`](../../sandbox-cluster.md) — 沙箱集群
- [`docs/sandbox-docker-backend.md`](../../sandbox-docker-backend.md) — Docker 后端
- [modules/02-agent.md § 6](./02-agent.md) — Agent 中 Skill / MCP 启用
- [modules/03-workflow.md § 7.5](./03-workflow.md) — 工作流中的 MCP Tool 节点
- [roles/03-admin.md § 3.3-3.5](../roles/03-admin.md) — Admin 视角

# 模块参考 · 系统设置（Settings）

> **入口**：`/platform/settings`（统一模态）
> 通过 `?section=` 切换所有配置
> **对应前端**：`frontend/src/views/settings/Settings.vue`（统一壳）+ 各子组件
> **配置文件**：`config/settingsAccess.ts`（最低角色表）

---

## 1. 设置模态导航
```

设置模态（左侧导航） ├── 通用 (viewer) ├── 模型 (admin) ├── Ollama (admin) ├── 网络搜索 (admin) ├── 聊天历史 (admin) ├── 向量库 (admin) ├── 解析器 (admin) ├── 存储 (admin) ├── 沙箱 (admin) ├── Skills (admin) ├── MCP (admin) ├── 记忆 (admin) ├── Env Var (viewer) ├── 工作区信息 (viewer 可看，编辑 owner) ├── 成员 (viewer 可看，操作 owner) ├── 我的记忆 (管理员) ├── 个人资料 (viewer) ├── 审计日志 (admin) ├── 集成 (viewer) │ ├── API Key Playground │ └── IM 落地页 ├── 待审批用户 (admin) ├── —— System Admin 专属 —— ├── 平台全局设置 (system admin) ├── 运行时队列 (system admin) ├── 平台 API Key (system admin) └── 平台审计日志 (system admin)

```
---

## 2. 通用设置（`section=general`）

| 项 | 说明 |
|---|---|
| UI 偏好 | 主题 / 语言 / 默认 KB |
| 默认 Agent | 全工作区默认 Agent（可被单个对话覆盖） |
| 通知偏好 | 邀请 / 审批通知 |

---

## 3. 模型（`section=models`）

> 详见 [roles/03-admin.md § 3.1](../roles/03-admin.md)

支持：`OpenAI / Azure OpenAI / Anthropic / DeepSeek / Qwen / Zhipu / Hunyuan /
Doubao / Gemini / MiniMax / NVIDIA / Novita / SiliconFlow / OpenRouter / Requesty /
LiteLLM / Ollama`

| 类型 | 用途 |
|---|---|
| Chat | LLM 对话 |
| Embedding | 向量嵌入 |
| Rerank | 重排 |
| VLM | 视觉语言模型 |
| ASR | 语音识别 |

详细清单：`config/builtin_models.yaml` + [`docs/BUILTIN_MODELS.md`](../../
BUILTIN_MODELS.md)。

### 3.1 创建模型

| 字段 | 说明 |
|---|---|
| 提供商 | 见上 |
| 模型名 | API 实际名称 |
| API Key / Endpoint | 凭证 |
| 类型 | Chat / Embedding / Rerank / VLM / ASR |

### 3.2 测试

- 「**测试连接**」→ 验证可达
- 显示延迟 / 模型元数据

### 3.3 使用详情

- 按模型看调用次数、token 用量
- 用于成本分析
```

```
---

## 4. Ollama（`section=ollama`）

- 本地 Ollama 服务配置
- 填入 endpoint（如 `http://localhost:11434`）
- 自动发现本地模型

---

## 5. 向量库（`section=vectorstore`）

> 详见 [`docs/使用其他向量数据库.md`](../../使用其他向量数据库.md)

| 后端 | 适用 |
|---|---|
| **pgvector**（默认） | 与 Postgres 一起，混合检索（pgvector + BM25） |
| **Qdrant** | 独立向量库 |
| **Milvus** | 大规模场景 |
| **Weaviate** | 多模态场景 |

填入连接信息 → 测试 → 保存。

---

## 6. 存储后端（`section=storage`）

> 对象存储（MinIO / S3 / COS / OSS）

| 后端 | 适用 |
|---|---|
| **MinIO**（默认） | 自托管 |
| **AWS S3** | 云端 |
| **腾讯 COS** | 腾讯云 |
| **阿里 OSS** | 阿里云 |

---

## 7. 解析引擎（`section=parser`）

- DocReader 服务连接（gRPC 端口 50051）
- 高级解析：MinerU endpoint（`MINERU_ENDPOINT`）
- 默认解析器选择

---

## 8. 网络搜索（`section=websearch`）

> 见 [`docs/添加新的网络搜索引擎.md`](../../添加新的网络搜索引擎.md)
```

```
支持：Bing / Tavily / Google / DuckDuckGo 等

填 API Key → 选择默认引擎。

---

## 9. 聊天历史（`section=chathistory`）

- 保留策略（天数 / 永久）
- 软删除 vs 硬删除
- 跨租户隔离

---

## 10. 沙箱（`section=sandbox`）

详见 [modules/05-skills-mcp.md § 3](./05-skills-mcp.md)。

---

## 11. Skills（`section=skills`）

详见 [modules/05-skills-mcp.md § 1](./05-skills-mcp.md)。

---

## 12. MCP（`section=mcp`）

详见 [modules/05-skills-mcp.md § 2](./05-skills-mcp.md)。

---

## 13. 长期记忆（`section=memory` + `section=mymemory`）

| 配置 | 说明 |
|---|---|
| 抽取策略 | 自动抽取哪些类（fact / preference / interest …） |
| 用户确认 | 抽取后是否需要用户确认 |
| 保留期 | 记忆有效期 |

`mymemory` 是个人级记忆查看 / 编辑。

---

## 14. Env Var（`section=envvars`）

- 工作区级环境变量
- 在 Skill 脚本中可访问
- 个人级 vs 工作区级

---
```

```
## 15. 工作区信息（`section=tenant`）

详见 [roles/04-owner.md § 5](../roles/04-owner.md)。

---

## 16. 成员（`section=members`）

详见 [roles/04-owner.md § 2-4](../roles/04-owner.md)。

---

## 17. 个人资料（`section=userprofile`）

- 头像 / 显示名
- 密码修改（8-32 字符）
- UI 偏好

---

## 18. 审计日志（`section=audit-log`）

- Admin+ 可看
- 详见 [roles/03-admin.md § 4](../roles/03-admin.md)

---

## 19. 集成（`section=integrations`）

> 子项：
> - API Key Playground（API Key 测试）
> - IM 落地页（10 个平台）

### 19.1 API Key Playground

- 创建 API Key → 测试调用
- 显示请求 / 响应

### 19.2 IM 集成

- 飞书 / 企微 / Lark / QQBot / Slack / Telegram / 钉钉 / Mattermost / 微信 / 云之家
- 详见 [`docs/IM集成开发文档.md`](../../IM集成开发文档.md)

---

## 20. 待审批用户（`section=pending-users`）

详见 [roles/03-admin.md § 2](../roles/03-admin.md)。

---
```

21. 系统设置（`section=system-global`，**仅 System Admin**）

key	说明
`default_storage_quota`	新租户默认配额
`enable_cross_tenant_access`	跨租户总开关
`enable_oauth_*`	OIDC 集成
`enable_im_*`	IM 总开关
`langfuse_*`	Langfuse 集成

---

22. 运行时队列（`section=runtime-queues`，**仅 System Admin**）

详见 [roles/05-system-admin.md § 5](../roles/05-system-admin.md)。

---

23. 平台 API Key（`section=platform-api-keys`，**仅 System Admin**）

详见 [roles/05-system-admin.md § 4](../roles/05-system-admin.md)。

---

24. 平台审计日志（`section=system-audit-log`，**仅 System Admin**）

- 跨租户查询
- 1 分钟滑动窗口去重

---

25. 关键配置 API

Method	路径	用途
GET	`/system/settings`	列设置
GET	`/system/settings/:key`	读单条
POST/PUT	`/system/settings/:key`	写单条
DELETE	`/system/settings/:key`	复位

> 来源：`system_setting.go` —— 强类型注册表（int / string / bool / string_list），Redis pub/sub 跨副本同步（`LUOSA_REDIS_NAMESPACE` 隔离多部署）。

---

26. 配置组织的最低视角表

> 来源：`frontend/src/config/settingsAccess.ts`

section	最低角色
`general`	viewer
`ollama`	admin
`models`	admin
`websearch`	admin
`chathistory`	admin
`vectorstore`	admin
`parser`	admin
`storage`	admin
`sandbox`	admin
`skills`	admin
`mcp`	admin
`memory`	admin
`tenant`	viewer（编辑 owner）
`members`	viewer（操作 owner）
`userprofile`	viewer
`envvars`	viewer
`pending-users`	admin
`audit-log`	admin
`integrations`	viewer

System Admin 专属（看不到的话说明不是 System Admin）：

- `system-global`
- `runtime-queues`
- `platform-api-keys`
- `system-audit-log`

---

27. 快捷入口最低视角（`SETTINGS_MANAGEMENT_SHORTCUT_MIN_ROLE`）

> 设置菜单的「管理快捷方式」（在用户菜单）要求更高：

快捷方式	最低角色
`members`	owner
`models`	admin
`skills`	admin

---

28. 进一步阅读

- [roles/03-admin.md](../roles/03-admin.md) — Admin 完整流程
- [roles/04-owner.md](../roles/04-owner.md) — Owner 完整流程
- [roles/05-system-admin.md](../roles/05-system-admin.md) — System Admin 完整流程
- [`docs/配置参考手册`](../../config-reference) — 完整配置项说明

模块参考 · Wiki 模式

```
> **对应后端**：`internal/application/service/wiki_page.go` + `wiki_ingest*.go` +
`wiki_linkify.go` + `wiki_lint.go`
> **对应前端**：`frontend/src/views/knowledge/wiki/*`
> **创建 KB 权限**：Contributor+（KB 类型选 Wiki）
> **编辑权限**：KB 创建者（`g.OwnedKBOrAdmin()`）

---

## 1. 概念

**Wiki 模式** = 把分散的文档**自动蒸馏**成结构化、可互链、可回滚的 Markdown 知识库。
```

原始文档（PDF / Word / Markdown …） ↓ wiki_ingest（并发摄取） ├── 行级 claim（不重不漏） ├── per-slug 锁 ├── 去重（wiki_ingest_dedup.go） ├── 分类法（wiki_ingest_taxonomy.go） └── 引用生成（wiki_ingest_cite.go） ↓ Wiki Page（Markdown + 互链 + 修订） ↓ Wiki 浏览器（前端） ├── 目录树 ├── 修订历史 + 行级 diff └── 一键回滚

```
---

## 2. 创建 Wiki 类型 KB

1. 进入 `/platform/knowledge-bases` → 「**+ 新建知识库**」
2. 类型选 **Wiki**
3. 选择默认 Markdown 模板（可选）
5. 创建后自动跳转到 Wiki 浏览器

---

## 3. 自动摄取（Agent Wiki Mode）

> 来源：`wiki_ingest*.go`（共 7 个文件，wiki_ingest.go 主 121 KB）

### 3.1 触发

- 手动：在 Wiki 浏览器 → 「**摄取**」按钮
- 自动：绑定 Agent → Agent 在回答时把知识点 push 到 Wiki

### 3.2 摄取流程
```

1. claimPendingList（行级 claim，防止并发重复）
2. withSlugLock（per-slug 锁）
3. 解析内容（拆分段落）
4. 去重（wiki_ingest_dedup.go，slug identity "wiki:identity:"）
5. 分类法（wiki_ingest_taxonomy.go）
6. 引用生成（wiki_ingest_cite.go）

7. 自动链接（wiki_linkify.go，识别 KB 内的相关 page 自动加链）
8. 内容质量检查（wiki_lint.go）
9. 写入（最大 32 KB 内容上限）

```
### 3.3 批量摄取（`wiki_ingest_batch.go`）

- 单批可摄取 40K 文档 KB
- asynq 任务调度
- 失败重试 + 续传

### 3.4 摄取源

- KB 内文档（Document 类型 KB）
- 外部数据源（Feishu Wiki / GitLab / Notion / Yuque）
- 手动上传

---

## 4. 手动编辑

> KB 创建者（`g.OwnedKBOrAdmin()`）

### 4.1 创建页面

1. Wiki 浏览器 → 「**+ 新页面**」
2. 填写：
   - **slug**（URL 友好，如 `product-overview`）
   - **标题**
   - **正文**（Markdown）
   - **类型**（page / index / redirect）
3. 保存

### 4.2 编辑页面

1. 点击页面 → 「**编辑**」
2. 修改 Markdown
3. 保存 → 自动生成新修订

### 4.3 文件夹操作（`WikiFolderActions.vue`）

- 新建文件夹
- 移动页面
- 删除文件夹（连带页面）

---

## 5. 修订历史

### 5.1 查看修订（`WikiRevisionDrawer.vue`）

1. 在 Wiki 页面 → 「**修订历史**」
2. 显示所有历史版本（时间、作者、commit message）
3. 点击版本 → 行级 diff
```

```
4. 「**回滚到此版本**」→ 一键恢复

### 5.2 修订 API

| Method | 路径 | 用途 |
|---|---|---|
| GET | `/wiki/pages/:slug/revisions` | 列表 |
| GET | `/wiki/pages/:slug/revisions/:rev_id` | 单个版本 |
| POST | `/wiki/pages/:slug/revisions/:rev_id/revert` | 回滚 |

---

## 6. 知识图谱（Wiki + Graph）

> Wiki 模式 + Knowledge Graph 启用后，会自动构建：

- **节点**：每个 Wiki 页面
- **边**：页面间的链接（自动 linkify + 手动）

### 6.1 交互式图谱

- Wiki 浏览器 → 「**图谱**」Tab
- 力导向图，可拖拽
- 节点大小 = 引用次数
- 边粗细 = 关联强度

### 6.2 图谱 API

| Method | 路径 | 用途 |
|---|---|---|
| GET | `/wiki/:kb_id/graph` | 取图谱数据 |

---

## 7. Wiki 与 Agent 协作

### 7.1 Wiki Mode Agent

- Agent 类型选 Wiki 模式 → 启用 `wiki_*` 工具
- Agent 在回答中可引用、写入 Wiki 页面（需要 sandbox）

### 7.2 wiki_* 工具（`internal/agent/tools/wiki_*`）

| 工具 | 作用 |
|---|---|
| `wiki_read_page` | 读 Wiki 页面 |
| `wiki_search` | 搜索 Wiki |
| `wiki_list_pages` | 列出页面 |
| `wiki_write_page` | 写页面（需权限） |
| `wiki_get_graph` | 取知识图谱 |
```

```
---

## 8. 共享 Wiki

- KB 共享给 Organization 时
- `OrgMemberRole.editor` 可编辑 Wiki 页面
- `OrgMemberRole.viewer` 只读

---

## 9. 内容质量检查（`wiki_lint.go`）

- 自动检查：
  - 链接是否失效
  - 标题层级是否规范
  - 内容是否过长
- 显示在页面顶部警告

---

## 10. 关键不变式

1. **Wiki 页面 32 KB 内容上限**——超长内容会拆页。
2. **slug identity 用 `wiki:identity:` 前缀**——确保唯一。
3. **per-slug 锁防并发写**。
4. **行级 claim 不重不漏**。
5. **修订可恢复**——保留完整历史。

---

## 11. 典型场景

### 场景 A：把客服 KB 升级为 Wiki

1. 现有 KB 是 Document 类型
2. 创建新 KB 类型 Wiki
3. 启用「**摄取**」→ 选择原 KB → 触发摄取任务
4. 自动生成结构化页面 + 互链
5. 手动调整 + 修订

### 场景 B：构建研发知识库

1. 创建 Wiki KB「研发手册」
2. 上传 Markdown / DOCX / 链接 GitLab
3. 自动摄取 + 分类（前端 / 后端 / 运维 / …）
4. Agent 在回答中自动 push 知识点
5. 知识图谱可视化

### 场景 C：跨部门协作 Wiki
```

```
1. Owner 创建 Organization
2. Wiki KB 共享给组织
3. 编辑者跨工作区协作
4. 修订历史可追溯

---

## 12. 进一步阅读

- [`docs/KnowledgeGraph.md`](../../KnowledgeGraph.md) — 知识图谱完整文档
- [`docs/开启知识图谱功能.md`](../../开启知识图谱功能.md) — 启用图谱
- [modules/01-knowledge-base.md § 5](./01-knowledge-base.md) — KB 设置
（GraphSettings / DataSourceSettings）
- [modules/02-agent.md § 7](./02-agent.md) — Agent 中 wiki_* 工具
- [roles/02-contributor.md § 3](../roles/02-contributor.md) — 创建第一个 Wiki KB
- [roles/04-owner.md § 7](../roles/04-owner.md) — Wiki 跨工作区共享
```